Apostila 3 — Visualização de Dados Econômicos (Teoria Completa)

Teoria da percepção, gráficos feitos com dados reais e o que evitar

Sumário

Apostila 3 — Visualização de Dados Econômicos (Teoria Completa) 1
Como Usar Esta Apostila 4
Capítulo 1 — Por que visualizar dados em economia 4
1.1. Um pouco de história: gráficos como argumento 4
1.2. matplotlib: a biblioteca padrão 5
1.3. A gramática de um gráfico: por que existem eixos 5
1.4. Anatomia de uma figura matplotlib 6
1.5. Duas interfaces: `pyplot` implícito vs. orientada a objetos 6
1.6. Exemplo Resolvido — Criando sua primeira figura com Figure e Axes explícitos 7
Capítulo 2 — Gráfico de linha e séries temporais 8
2.1. Por que linha é o padrão para dados ao longo do tempo 8
2.2. Sintaxe e parâmetros de `ax.plot()` 9
2.3. Densidade de dados e o problema do "espaguete" 9
2.4. Cor como comunicação, não decoração 10
2.5. Exemplo Resolvido — Da tabela ao gráfico: decidindo o que contar 10
Capítulo 3 — Comparando múltiplas séries: mesma escala, dois eixos, subplots 12
3.1. Mesma escala: o caso mais simples e mais honesto 12
3.2. Dois eixos Y (`twinx`) — poderoso, mas perigoso 12
3.3. Subplots: a alternativa mais honesta a dois eixos Y 13
3.4. Grade de subplots (mais de dois gráficos) 13
3.5. Exemplo Resolvido — Escolhendo entre `twinx` e subplots 14
Capítulo 4 — Gráfico de barras: comparação de categorias 16
4.1. Barra vs. linha: uma decisão sobre o tipo do eixo X 16
4.2. Comprimento, não área: por que a barra funciona 16
4.3. A regra do eixo zero em gráficos de barra 16
4.4. Barras horizontais 17
4.5. Barras agrupadas e empilhadas 17
4.6. Exemplo Resolvido — Escolhendo e defendendo o tipo de gráfico 18
Capítulo 5 — Dispersão (scatter): relação entre variáveis 19
5.1. O que um scatter revela que uma linha ou barra não revelam 19
5.2. Overplotting e transparência 20
5.3. Linha de tendência — e o lembrete mais importante deste capítulo 20
5.4. Anotando pontos específicos 20
5.5. Exemplo Resolvido — Curva de Phillips: montando e interpretando o scatter 21
Capítulo 6 — Histograma: a forma da distribuição 22
6.1. O que um histograma mostra 22
6.2. A escolha do número de *bins* não é neutra 23
6.3. Assimetria (skewness) e o que ela significa economicamente 23
6.4. Sobrepondo distribuições para comparação 24
6.5. Exemplo Resolvido — Histograma e boxplot lado a lado 24
Capítulo 7 — Boxplot: resumo estatístico visual 25
7.1. Anatomia de um boxplot 25
7.2. Boxplot vs. histograma: quando usar cada um 26
7.3. Comparando grupos com boxplot 26
7.4. Exemplo Resolvido — Diagnosticando outliers antes de descartá-los 27
Capítulo 8 — Heatmap e matrizes de correlação 28
8.1. Codificando valores por cor: sequencial vs. divergente 28
8.2. A armadilha do colormap "arco-íris" (jet) 29

8.3. Construindo um heatmap passo a passo 29
8.4. Exemplo Resolvido — Corrigindo um heatmap mal projetado 29
Capítulo 9 — Teoria da percepção visual e escolha de cor 31
9.1. Retomando a hierarquia de Cleveland-McGill 31
9.2. Três famílias de paleta de cor 31
9.3. Daltonismo e paletas seguras 32
9.4. Redundância de codificação: nunca dependa só da cor 32
9.5. Exemplo Resolvido — Auditando um gráfico quanto à acessibilidade de cor 32
Capítulo 10 — Gráficos enganosos ("lying with charts") 33
10.1. Por que este capítulo existe 33
10.2. Eixo Y cortado em gráfico de barras 34
10.3. Área e raio em vez de valor: o erro dos "círculos proporcionais" 34
10.4. Gráfico de pizza: ângulo é uma codificação fraca 34
10.5. Eixos duplos manipulados 35
10.6. Escala logarítmica sem aviso 35
10.7. Checklist de honestidade visual 35
10.8. Exemplo Resolvido — Auditoria de um gráfico suspeito 36
Capítulo 11 — Anotações, texto e storytelling com dados econômicos 37
11.1. Títulos que contam uma história 37
11.2. Anotando eventos: `axvline` e `annotate` juntos 37
11.3. Créditos de fonte: uma prática de rigor, não de estilo 38
11.4. Exemplo Resolvido — Transformando um gráfico "cru" em um gráfico que conta uma história 38
Capítulo 12 — Personalização avançada e gráficos publication-ready 40
12.1. `rcParams`: configurando o estilo padrão uma única vez 40
12.2. Removendo bordas desnecessárias (spines) 40
12.3. Formatando eixos de data, milhar e percentual 40
12.4. Exemplo Resolvido — O gráfico publication-ready completo 40
Capítulo 13 — seaborn: gráficos estatísticos de alto nível 42
13.1. A relação entre seaborn e matplotlib 42
13.2. Funções de eixo (*axes-level*) vs. de figura (*figure-level*) 42
13.3. Regressão com intervalo de confiança 43
13.4. `pairplot`: todas as correlações de uma vez 43
13.5. Boxplot com seaborn: menos código, mesmo resultado 43
13.6. Exemplo Resolvido — Recriando um gráfico matplotlib em seaborn e comparando o código 43
Capítulo 14 — Salvando e exportando gráficos 45
14.1. Raster vs. vetorial: a distinção que mais importa 45
14.2. DPI: o que realmente significa 45
14.3. `bbox_inches="tight"` e recorte de bordas 46
14.4. Transparência de fundo 46
14.5. Exemplo Resolvido — Escolhendo o formato certo para cada destino 46
Capítulo 15 — Boas práticas: checklist final de um gráfico econômico 47
Capítulo 16 — Exercícios para Executar (na mão) 47
Exercício 1 — Identifique o erro de design 47
Exercício 2 — Escolha o tipo de gráfico 47
Exercício 3 — Preveja antes de rodar 48
Exercício 4 — Diagnóstico de gráfico enganoso 48
Exercício 5 — Corrija a paleta 48
Exercício 6 — Redesenhando um título 48
Capítulo 17 — Exercícios para Executar (em código) 48
Exercício 1 — Primeira figura orientada a objetos 48
Exercício 2 — Dois eixos Y 48
Exercício 3 — Subplots como alternativa honesta 48
Exercício 4 — Barras com regra do eixo zero 49

Exercício 5 — Barras horizontais ordenadas 49
Exercício 6 — Curva de Phillips 49
Exercício 7 — Mesmo scatter com seaborn 49
Exercício 8 — Histograma com bins alternativos 49
Exercício 9 — Histograma com média e mediana 49
Exercício 10 — Histograma e boxplot lado a lado 49
Exercício 11 — Boxplot comparando países 49
Exercício 12 — Heatmap de correlação correto 49
Exercício 13 — Heatmap com colormap errado (para comparar) 49
Exercício 14 — Redundância de codificação 49
Exercício 15 — Gráfico de barras enganoso vs. honesto 49
Exercício 16 — Anotando um evento histórico 49
Exercício 17 — Gráfico publication-ready completo 49
Exercício 18 — Bubble chart com área corrigida 50
Exercício 19 — Exportando para três destinos 50
Exercício 20 — Pairplot exploratório 50
Exercício 21 — Projeto integrador: painel de 4 gráficos 50
Capítulo Final — Resumo 50
Capítulo 18 — Erros Comuns e Teste Rápido 51
18.1. Os erros de design que mais prejudicam um gráfico econômico 51
18.2. Teste rápido: verdadeiro ou falso (com gabarito) 51
Próxima apostila 51

Como Usar Esta Apostila

A terceira apostila da série trata de **comunicar dados**. Programar um gráfico é fácil; decidir *qual gráfico* responde à pergunta, evitar que ele engane e deixá-lo legível para o leitor é a parte difícil — e é onde esta apostila gasta seu tempo. Pressupõe as Apostilas 1 e 2.

Ao final de cada capítulo, um bloco de **aprofundamento** traz um gráfico *gerado pelo código mostrado*, com **dados reais deste repositório**: a Selic meta diária (`selic_meta.json`), o IPCA acumulado em 12 meses (`ipca_12m.json`), a abertura do IPCA por grupos (`ipca_abertura_grupos.csv`) e a regra de Taylor (`taylor_rule_series.csv`). As figuras ficam na pasta `figuras03/`. Caixas coloridas:

Exemplo resolvido — modelo
Um gráfico construído passo a passo, com o código.

Intuição
A ideia por trás da regra, sem jargão.

Aprofundando a teoria — modelo
O que a ciência da percepção diz sobre o design.

Atenção — erro comum
Um erro clássico de design (que engana ou confunde o leitor).

No mercado brasileiro
Como a prática profissional de analistas e jornalistas de dados lida com isso.

```
import matplotlib.dates as mdates

def ler_bcb(caminho):
    d = pd.read_json(caminho, dtype=False)
    d["data"] = pd.to_datetime(d["data"], format="%d/%m/%Y")
    return d.set_index("data")["valor"].astype(float)

sel3, ip3 = ler_bcb("selic_meta.json"), ler_bcb("ipca_12m.json")
gr3 = pd.read_csv("ipca_abertura_grupos.csv", parse_dates=["data"], index_col="data")
tay3 = pd.read_csv("taylor_rule_series.csv", parse_dates=["data"], index_col="data")
FONTE3 = "Fonte: Banco Central do Brasil (SGS); elaboração própria."
OKABE = ["#E69F00", "#56B4E9", "#009E73", "#D55E00", "#0072B2", "#CC79A7", "#000000"]   #
paleta segura para daltônicos
print(f"Selic: {len(sel3)} dias | IPCA 12m: {len(ip3)} meses | grupos: {gr3.shape} |
Taylor: {tay3.shape}")
```

```
Selic: 1355 dias | IPCA 12m: 43 meses | grupos: (43, 6) | Taylor: (43, 6)
```

Capítulo 1 — Por que visualizar dados em economia

1.1. Um pouco de história: gráficos como argumento

A ideia de representar dados econômicos em gráficos não é recente. Em 1786, o escocês William Playfair publicou o *Commercial and Political Atlas*, onde inventou (praticamente sozinho) o gráfico de linha para séries temporais e o gráfico de barras para comparação de categorias — usando exatamente o problema que ainda hoje motiva este capítulo: mostrar a balança comercial da Inglaterra ao longo do tempo de um jeito que nenhuma

tabela de números conseguiria comunicar tão rápido. Playfair escreveu, já em 1786, que uma tabela "cansa a atenção e confunde a memória", enquanto um gráfico bem feito "fala aos olhos".

Mais de um século depois, Charles Minard desenhou o que Edward Tufte (estatístico e teórico da visualização de dados) chamou de "talvez o melhor gráfico estatístico já feito": um único desenho que mostra, simultaneamente, o tamanho do exército de Napoleão na invasão da Rússia em 1812, sua posição geográfica, a direção do movimento (avanço vs retirada) e a temperatura durante a retirada. Seis variáveis, um gráfico, nenhuma legenda complicada — e o colapso do exército francês se torna visualmente evidente sem que nenhum número precise ser lido isoladamente.

Para um economista, a lição por trás dessas duas histórias é a mesma: **um gráfico não é uma decoração colocada depois da análise — é parte do argumento**. Um bom gráfico de IPCA acumulado não apenas "ilustra" que a inflação subiu; ele revela em que mês a aceleração começou, se houve reversão, e como o período se compara historicamente — informação que está tecnicamente presente numa tabela de 300 linhas, mas que ninguém consegue extrair olhando números.

1.2. matplotlib: a biblioteca padrão

matplotlib é a biblioteca de gráficos mais usada do ecossistema científico de Python. Foi criada por John D. Hunter em 2003, inicialmente para reproduzir, em Python, o tipo de gráfico que o MATLAB gerava (daí o nome) — o que explica por que sua API tem uma "camada de compatibilidade" chamada `pyplot`, que imita o estilo de comandos do MATLAB, e uma API mais moderna, orientada a objetos, que é a recomendada para qualquer script que você pretenda reaproveitar.

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Praticamente todas as outras bibliotecas de visualização do ecossistema Python — seaborn, pandas `.plot()`, statsmodels — são construídas **em cima** de matplotlib: elas simplificam a sintaxe para casos comuns, mas por baixo continuam gerando os mesmos objetos `Figure` e `Axes` que veremos a seguir. Entender matplotlib bem, portanto, não é aprender "mais uma biblioteca" — é aprender a fundação sobre a qual quase tudo o resto se apoia.

1.3. A gramática de um gráfico: por que existem eixos

Um erro comum de quem começa é pensar em "fazer um gráfico" como uma ação única (`plt.plot(dados)` e pronto). Na prática, todo gráfico estatístico é composto por peças com papéis bem definidos, e entender essas peças separadamente é o que permite compor gráficos complexos (dois eixos Y, múltiplos subplots, anotações) sem se perder.

Essa ideia tem um nome formal: **gramática de gráficos** (*grammar of graphics*), formalizada pelo estatístico Leland Wilkinson em 1999 e popularizada pelo pacote `ggplot2` (linguagem R). A ideia central é que qualquer gráfico estatístico pode ser decomposto em camadas independentes:

Camada	Pergunta que responde	Exemplo econômico
Dados	O que estou plotando?	A série mensal do IPCA
Mapeamento (aesthetic)	Qual variável vai em qual eixo/cor/tamanho?	Data no eixo X, variação % no eixo Y
Geometria (geom)	Que forma visual representa cada observação?	Linha, barra, ponto, área
Escala	Como o valor dos dados vira posição na tela?	Linear, logarítmica, eixo cortado ou não
Eixos e grade	Como o leitor lê as escalas?	Rótulos, ticks, grid
Anotações	O que preciso destacar explicitamente?	"Início da pandemia", "Meta de inflação"

matplotlib não impõe essa gramática de forma tão explícita quanto `ggplot2`, mas os objetos que ele expõe — `Figure`, `Axes`, `Artist` — mapeiam diretamente para essas camadas. Pensar dessa forma, desde o início, evita a

armadilha de tratar cada tipo de gráfico (linha, barra, scatter) como algo isolado: na verdade, todos compartilham a mesma estrutura, e só muda a "geometria" usada para representar os dados.

1.4. Anatomia de uma figura matplotlib

matplotlib organiza tudo em uma hierarquia de objetos. Os dois mais importantes, que você vai usar em literalmente todo gráfico deste material, são:

- **`Figure`**: a "folha de papel" inteira — o objeto que você salva em arquivo (`fig.savefig(...)`). Uma `Figure` pode conter um ou vários `Axes`.
- **`Axes`**: uma **área de plotagem individual** dentro da figura — com seu próprio eixo X, eixo Y, título, grade. É dentro de um `Axes` que os dados de fato são desenhados (`ax.plot(...)`, `ax.bar(...)`).

Um erro de vocabulário muito comum (inclusive em fóruns e documentação traduzida) é confundir **Axes** (a área de plotagem, no singular "um Axes") com **axis** (um eixo individual — X ou Y). Uma `Figure` com dois gráficos lado a lado tem **dois objetos `Axes`**, e cada `Axes` tem **dois `axis`** (X e Y). Essa distinção de nomenclatura não é só semântica — ela aparece o tempo todo no código (`ax.xaxis`, `ax.yaxis` vs. simplesmente `ax`).

```
Figure                              <- a "folha" inteira, salva com fig.savefig()
 └── Axes (ax)                      <- uma área de plotagem
     ├── xaxis (eixo X)             <- rótulos, ticks, escala do eixo horizontal
     ├── yaxis (eixo Y)             <- rótulos, ticks, escala do eixo vertical
     ├── title                      <- título do Axes
     ├── artists (linhas, barras)   <- os dados desenhados de fato
     └── legend                     <- legenda (se houver)
```

Uma `Figure` pode conter vários `Axes` — é exatamente isso que acontece quando você faz `plt.subplots(2, 2)`: uma figura com uma grade de 2×2 = 4 áreas de plotagem independentes, cada uma com seu próprio `ax`.

1.5. Duas interfaces: `pyplot` implícito vs. orientada a objetos

matplotlib oferece duas formas de escrever o mesmo gráfico, e a diferença entre elas causa bastante confusão para quem aprende olhando exemplos de fontes diferentes na internet.

Interface implícita (`pyplot`, estilo MATLAB) — mantém um "gráfico atual" escondido, e cada comando `plt.algumacoisa()` opera sobre ele:

```
plt.plot(ipca.index, ipca["IPCA"])
plt.title("IPCA")
plt.ylabel("Variação (%)")
plt.show()
```

Interface orientada a objetos — você cria explicitamente os objetos `Figure` e `Axes`, e chama métodos diretamente sobre eles:

```
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(ipca.index, ipca["IPCA"])
ax.set_title("IPCA")
ax.set_ylabel("Variação (%)")
```

A diferença parece cosmética, mas não é. A interface implícita depende de um estado global escondido ("qual é o gráfico atual?"), o que funciona bem para um gráfico único e rápido no terminal, mas se torna frágil e confuso assim que você tem mais de um `Axes` na mesma figura (qual dos dois é o "atual"?). A interface orientada a objetos é explícita: cada variável (`ax1`, `ax2`) aponta exatamente para o `Axes` que você quer modificar, sem ambiguidade — o mesmo princípio de "explícito é melhor que implícito" do Zen of Python, visto na Apostila 1.

Recomendação seguida em todo este material: use sempre a interface orientada a objetos (`fig, ax = plt.subplots()`), mesmo para gráficos simples de um único `Axes`. O custo de escrever duas linhas a mais (`fig, ax = plt.subplots()`) é irrelevante perto do ganho de previsibilidade quando o gráfico crescer — e ele quase sempre cresce, seja para adicionar um segundo eixo Y, seja para criar subplots.

1.6. Exemplo Resolvido — Criando sua primeira figura com Figure e Axes explícitos

Enunciado: usando a biblioteca `bcb` (já usada nas apostilas anteriores) para baixar a série do IPCA (código SGS 433) desde 2020, crie uma figura com `plt.subplots()`, plote a série, adicione título e rótulo do eixo Y, e salve em disco com 150 DPI. Identifique, no código, qual linha cria a `Figure`, qual cria o `Axes`, e qual efetivamente desenha os dados.

Solução comentada:

```
from bcb import sgs
import matplotlib.pyplot as plt

ipca = sgs.get({"IPCA": 433}, start="2020-01-01")

# Esta linha cria DOIS objetos ao mesmo tempo: a Figure (fig) e o Axes (ax)
fig, ax = plt.subplots(figsize=(12, 5))

# Esta linha desenha os dados DENTRO do Axes — ainda não existe nada
# na figura antes desta chamada
ax.plot(ipca.index, ipca["IPCA"], color="crimson", linewidth=1.5)

# Estas linhas configuram o Axes (título, rótulo) — não desenham dados novos
ax.set_title("IPCA — Variação Mensal (%)", fontsize=14, fontweight="bold")
ax.set_ylabel("Variação mensal (%)")

# fig.savefig() opera sobre a FIGURE inteira, não sobre o Axes
fig.savefig("ipca_2020.png", dpi=150, bbox_inches="tight")
plt.close(fig)  # libera a memória usada pela figura
```

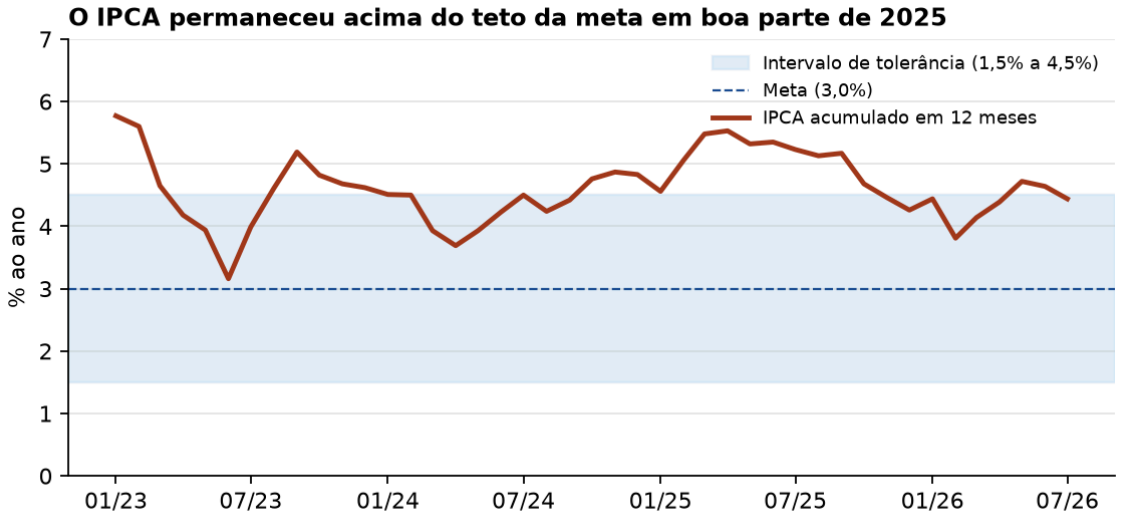
O raciocínio: `plt.subplots()` é uma função de conveniência que cria a `Figure` e já cria um (ou mais) `Axes` dentro dela, retornando os dois como uma tupla — por isso `fig, ax = plt.subplots()`. `ax.plot(...)` não cria nada novo na hierarquia: ele adiciona um "artista" (a linha desenhada) dentro do `Axes` que já existia. Por fim, `fig.savefig(...)` é chamado na `Figure`, porque salvar em arquivo é uma operação sobre a folha inteira — inclusive quando ela contém vários `Axes` lado a lado. `plt.close(fig)` é um hábito recomendado em scripts que geram muitos gráficos em sequência (por exemplo, um relatório automático mensal): sem fechar, cada figura permanece na memória, o que eventualmente deixa o script mais lento ou trava com "too many open figures".

> **Aprofundando a teoria — tinta de dados, memória visual e o que a figura precisa ter**
> Edward Tufte propôs o conceito de **razão tinta-dados**: a fração da "tinta" do gráfico que realmente representa dados. Grades pesadas, molduras, fundos coloridos e efeitos 3D consomem tinta sem informar: o gráfico ideal tem o máximo de dados e o mínimo de enfeite. Mas há um piso: a figura precisa ser **autoexplicativa**, com *título que diz a conclusão*, eixos com unidade, legenda ou rótulos diretos, e fonte dos dados. A figura abaixo reúne esses elementos, ao mostrar o IPCA em 12 meses contra o intervalo do regime de metas (3% de meta, com tolerância de 1,5 ponto percentual):

```
fig, ax = plt.subplots(figsize=(7.2, 3.6))
ax.axhspan(1.5, 4.5, color="#cfe3f5", alpha=0.6, label="Intervalo de tolerância (1,5% a
4,5%)")
ax.axhline(3.0, color="#1d4f91", linestyle="--", linewidth=1, label="Meta (3,0%)")
ax.plot(ip3.index, ip3.values, color="#a33a1c", linewidth=2.2, label="IPCA acumulado em 12
meses")
ax.set_title("O IPCA permaneceu acima do teto da meta em boa parte de 2025", loc="left",
fontweight="bold", fontsize=11)
ax.set_ylabel("% ao ano"); ax.set_ylim(0, 7); ax.grid(axis="y", alpha=0.3)
ax.xaxis.set_major_formatter(mdates.DateFormatter("%m/%y"))
ax.legend(loc="upper right", fontsize=8, frameon=False)
for lado in ("top", "right"):
    ax.spines[lado].set_visible(False)
```

```
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/anatomia_ipca.png", dpi=160)
plt.close(fig)
print("Meses com IPCA 12m acima do teto:", int((ip3 > 4.5).sum()), "de", len(ip3))
```

```
Meses com IPCA 12m acima do teto: 24 de 43
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 1: Anatomia de uma figura: título com a mensagem, faixa de referência, eixo com unidade, legenda, fonte e eliminação de bordas desnecessárias.

Capítulo 2 — Gráfico de linha e séries temporais

2.1. Por que linha é o padrão para dados ao longo do tempo

A escolha do tipo de gráfico não é estética — é uma decisão sobre **qual comparação você quer que o leitor faça instantaneamente**. Em 1984, os estatísticos William Cleveland e Robert McGill publicaram um estudo experimental hoje clássico, medindo o quão precisamente o olho humano consegue comparar valores codificados de formas visuais diferentes. O resultado — conhecido como a **hierarquia de Cleveland-McGill** — ordena as codificações da mais precisa para a menos precisa:

1. Posição ao longo de uma escala comum (dois pontos no mesmo eixo)
2. Posição ao longo de escalas não alinhadas
3. Comprimento
4. Inclinação (*slope*)
5. Ângulo
6. Área
7. Volume
8. Cor / saturação

Um gráfico de linha para uma série temporal explora exatamente os dois primeiros itens dessa lista — **posição** (onde está o ponto no eixo Y para cada mês) e **inclinação** (a taxa de variação entre um mês e o seguinte é literalmente a inclinação do segmento de reta). É por isso que uma linha comunica tão bem "o IPCA acelerou entre março e abril": o olho lê a inclinação da reta quase instantaneamente, sem precisar comparar dois números.

Além da precisão perceptual, a linha carrega uma **implicação semântica**: ao conectar um ponto ao seguinte, você está dizendo visualmente "existe uma continuidade entre essas observações" — o que é verdadeiro para

uma série temporal (o IPCA de fevereiro é seguido pelo IPCA de março, numa linha do tempo contínua), mas seria **enganoso** para categorias sem ordem natural (ligar com uma linha o PIB de "Brasil" ao de "Chile" sugeriria uma continuidade que não existe entre os dois países — veremos isso no Capítulo 4, sobre por que barra, não linha, é o gráfico certo para categorias).

2.2. Sintaxe e parâmetros de `ax.plot()`

```
from bcb import sgs

ipca = sgs.get({"IPCA": 433}, start="2020-01-01")

fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(
    ipca.index, ipca["IPCA"],
    color="crimson",      # cor: nome ("red"), hex ("#FF5733") ou tupla RGB
    linewidth=1.5,        # espessura da linha
    linestyle="-",        # "-" sólida, "--" tracejada, ":" pontilhada, "-." traço-ponto
    marker="o",           # marcador em cada ponto: "o" círculo, "^" triângulo, "s"
quadrado
    markersize=4,
    alpha=0.9,            # transparência (0 = invisível, 1 = opaco)
    label="IPCA"          # texto que aparece na legenda, se ax.legend() for chamado
)

ax.set_title("IPCA — Variação Mensal (%)", fontsize=14, fontweight="bold")
ax.set_ylabel("Variação mensal (%)")
ax.set_xlabel("")
ax.grid(True, alpha=0.3)

fig.tight_layout()
fig.savefig("ipca_2020.png", dpi=150)
plt.close(fig)
```

`fig.tight_layout()` merece nota à parte: ele recalcula automaticamente as margens da figura para que títulos, rótulos e legendas não fiquem cortados nas bordas — um problema extremamente comum ao salvar figuras com título longo ou rótulos de eixo grandes. É um hábito recomendado chamar `tight_layout()` (ou usar `bbox_inches="tight"` em `savefig`) em praticamente todo gráfico salvo em arquivo.

2.3. Densidade de dados e o problema do "espaguete"

Uma armadilha comum ao plotar séries longas: quando o número de pontos no eixo X é muito grande em relação à largura disponível (por exemplo, plotar dados diários de câmbio dos últimos 20 anos numa figura pequena), a linha vira uma massa visualmente ilegível de oscilações — informalmente chamada de "gráfico espaguete". Três soluções, não excludentes:

- **Aumentar a figura** (`figsize=(16, 6)` em vez de `(6, 4)`), ganhando resolução horizontal.
- **Agregar os dados** antes de plotar (média mensal em vez de diária, por exemplo), reduzindo o número de pontos sem perder a tendência principal — isso é feito com `resample()` do pandas, visto na Apostila 2.
- **Suavizar com média móvel**, plotando a série bruta com baixa opacidade (`alpha=0.3`) e a média móvel por cima com linha cheia — uma técnica comum em gráficos de câmbio e bolsa, que mostra tanto a volatilidade quanto a tendência.

```
cambio = sgs.get({"USDBRL": 1}, start="2015-01-01")
cambio["media_movel_30d"] = cambio["USDBRL"].rolling(30).mean()

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(cambio.index, cambio["USDBRL"], color="steelblue", alpha=0.25, linewidth=0.8,
label="Diário")
ax.plot(cambio.index, cambio["media_movel_30d"], color="steelblue", linewidth=2,
```

```
label="Média móvel 30 dias")
ax.legend()
ax.set_title("Câmbio USD/BRL — Diário e Média Móvel")
fig.tight_layout()
```

2.4. Cor como comunicação, não decoração

Escolher uma cor não é um detalhe estético isolado — em um gráfico econômico, a cor frequentemente carrega significado que o leitor já traz consigo antes mesmo de ler os rótulos. Vermelho tende a ser lido como "alerta" ou "negativo" (inflação alta, PIB em queda); verde ou azul como "neutro" ou "positivo". Usar essas associações **a favor** do gráfico (vermelho para uma série de inflação, por exemplo) reduz o esforço cognitivo do leitor; usá-las contra a intuição (verde para uma crise) exige que o leitor pare, leia a legenda com atenção, e desfaça uma primeira impressão errada — o que é o oposto do objetivo de um gráfico. Voltaremos a esse tema com mais profundidade teórica no Capítulo 9.

2.5. Exemplo Resolvido — Da tabela ao gráfico: decidindo o que contar

Enunciado: você tem a série mensal do IPCA de 2015 a 2026. O objetivo do gráfico é comunicar "a inflação estava descontrolada em 2015-2016, foi controlada depois, e voltou a acelerar em 2021". Escreva o código do gráfico e explique, em prosa, quais decisões de design (cor, anotação, linha de referência) ajudam a contar exatamente essa história — e não outra.

Solução comentada:

```
from bcb import sgs
import matplotlib.pyplot as plt

ipca = sgs.get({"IPCA": 433}, start="2015-01-01")

fig, ax = plt.subplots(figsize=(13, 5))

ax.plot(ipca.index, ipca["IPCA"], color="crimson", linewidth=1.3)

# Linha de referência: a meta de inflação mensal implícita (~0.29% a.m. equivale a 3.5%
a.a.)
ax.axhline(0.29, color="gray", linestyle="--", linewidth=1, label="Meta implícita (~3,5%
a.a.)")

ax.set_title("IPCA Mensal — Da Crise de 2015-16 à Retomada de 2021", fontsize=13,
fontweight="bold")
ax.set_ylabel("Variação mensal (%)")
ax.legend(frameon=False)
ax.grid(True, alpha=0.25)
fig.tight_layout()
```

O raciocínio de design: o título **não é genérico** ("IPCA 2015-2026") — ele já entrega a interpretação que o gráfico deve confirmar visualmente, funcionando como uma "legenda de jornal" que orienta a leitura (voltaremos a esse ponto no Capítulo 11, sobre storytelling). A linha de referência horizontal (`axhline`) dá ao leitor uma régua contra a qual comparar cada mês, sem precisar decorar o valor da meta. A cor vermelha reforça a leitura de "tema de alerta" (inflação), consistente com a Seção 2.4. Nenhuma dessas escolhas altera os dados — mas todas alteram a velocidade e a direção com que o leitor os interpreta, o que é exatamente o papel de um gráfico bem desenhado.

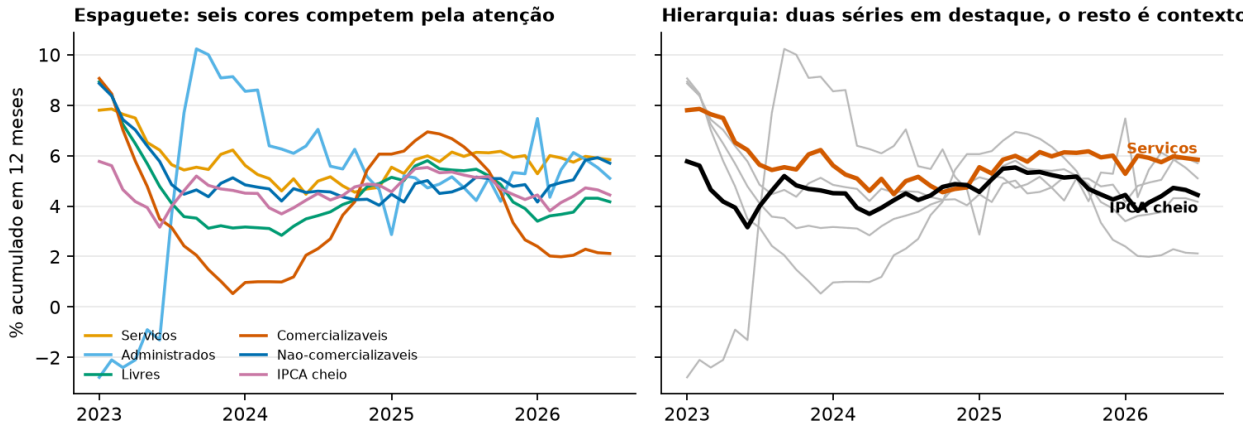
> **Aprofundando a teoria — como o cérebro lê um gráfico de linhas: Gestalt e o problema do espaguete**
> O sistema visual agrupa elementos por **proximidade, semelhança e continuidade** (princípios da Gestalt). Em um gráfico de linhas, a *continuidade* faz com que sigamos cada linha de um lado ao outro — até o ponto em que várias se cruzam, quando a trajetória se perde. Com mais de 4 ou 5 séries coloridas, o gráfico vira um "espaguete". A solução é **hierarquizar**: destacar em cor a série que importa e deixar as

demais em **cinza** como contexto, ou separar em painéis (*small multiples*). A figura mostra as duas versões dos mesmos dados:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9.5, 3.6), sharey=True)
cores_todas = dict(zip(gr3.columns, OKABE))
for col in gr3.columns:
    ax1.plot(gr3.index, gr3[col], label=col, linewidth=1.6, color=cores_todas[col])
ax1.set_title("Espaguete: seis cores competem pela atenção", loc="left", fontsize=10,
fontweight="bold")
ax1.legend(fontsize=7, frameon=False, ncol=2, loc="lower left")

for col in gr3.columns:
    destaque = col in ("Servicos", "IPCA cheio")
    ax2.plot(gr3.index, gr3[col], linewidth=2.4 if destaque else 1.0,
             color={"Servicos": "#D55E00", "IPCA cheio": "#000000"}.get(col, "#bbbbbb"),
zorder=3 if destaque else 1)
ax2.text(gr3.index[-1], gr3["Servicos"].iloc[-1] + 0.25, "Serviços", color="#D55E00",
fontsize=8, ha="right", fontweight="bold")
ax2.text(gr3.index[-1], gr3["IPCA cheio"].iloc[-1] - 0.7, "IPCA cheio", color="#000000",
fontsize=8, ha="right", fontweight="bold")
ax2.set_title("Hierarquia: duas séries em destaque, o resto é contexto", loc="left",
fontsize=10, fontweight="bold")
for ax in (ax1, ax2):
    ax.xaxis.set_major_locator(mdates.YearLocator());
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y")); ax.grid(axis="y", alpha=0.25)
    ax.spines[["top", "right"]].set_visible(False)
ax1.set_ylabel("% acumulado em 12 meses")
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/espaguete_vs_destaque.png", dpi=160)
plt.close(fig)
print("Séries no gráfico:", len(gr3.columns), "| em destaque na versão 2:", 2)
```

```
Séries no gráfico: 6 | em destaque na versão 2: 2
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 2: À esquerda, o "espaguete"; à direita, a mesma informação com hierarquia visual (cor para o que importa, cinza para o contexto) e rótulos diretos no lugar de legenda.

Capítulo 3 — Comparando múltiplas séries: mesma escala, dois eixos, subplots

3.1. Mesma escala: o caso mais simples e mais honesto

Quando duas séries estão na mesma unidade (por exemplo, IPCA e SELIC, ambas em % ao mês ou % ao ano, com magnitudes parecidas), a forma mais direta e honesta de compará-las é plotá-las no **mesmo eixo Y**:

```
fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(ipca.index, ipca["IPCA"], color="crimson", label="IPCA")
ax.plot(selic.index, selic["SELIC"], color="navy", label="SELIC")

ax.legend()
ax.set_title("IPCA vs SELIC")
ax.grid(True, alpha=0.3)
fig.tight_layout()
```

Isso funciona bem porque a distância vertical entre as duas linhas, em qualquer ponto do eixo X, corresponde diretamente ao juro real aproximado (SELIC menos IPCA) — uma leitura direta e sem distorção.

3.2. Dois eixos Y (`twinx`) — poderoso, mas perigoso

Quando duas séries têm **unidades ou magnitudes muito diferentes** (por exemplo, IPCA em % versus o índice Ibovespa em pontos, na casa de 100 mil), colocá-las no mesmo eixo Y torna uma delas visualmente "achatada" — invisível perto da outra. A solução técnica é um segundo eixo Y, com sua própria escala, compartilhando o mesmo eixo X:

```
fig, ax1 = plt.subplots(figsize=(12, 5))

ax1.plot(ipca.index, ipca["IPCA"], color="crimson", linewidth=1.5, label="IPCA")
ax1.set_ylabel("IPCA (%)", color="crimson")
ax1.tick_params(axis="y", labelcolor="crimson")

ax2 = ax1.twinx()  # cria um segundo Axes que COMPARTILHA o eixo X com ax1
ax2.plot(selic.index, selic["SELIC"], color="navy", linewidth=1.5, label="SELIC")
ax2.set_ylabel("SELIC (%)", color="navy")
ax2.tick_params(axis="y", labelcolor="navy")

ax1.set_title("IPCA e SELIC — 2020 a 2026", fontsize=14)
ax1.grid(True, alpha=0.3)
fig.tight_layout()
fig.savefig("ipca_selic.png", dpi=150)
plt.close(fig)
```

`ax1.twinx()` cria um **novo `Axes`** (`ax2`) que ocupa exatamente o mesmo espaço de `ax1`, mas com um eixo Y independente à direita — os dois `Axes` compartilham apenas o eixo X.

A armadilha central de dois eixos Y, e o motivo pelo qual estatísticos como Tufte recomendam evitá-los sempre que possível: **a escala de cada eixo é uma escolha arbitrária de quem desenhou o gráfico**, e escolhas diferentes de escala podem fazer duas séries **sem nenhuma relação real** parecerem visualmente correlacionadas — ou fazer duas séries fortemente correlacionadas parecerem independentes. Como não existe uma forma "objetiva" de alinhar duas escalas diferentes, quem monta o gráfico tem, na prática, o poder de "ajustar" a impressão visual só escolhendo os limites de cada eixo. Isso é explorado deliberadamente em gráficos enganosos (voltaremos a esse ponto, com um exemplo concreto, no Capítulo 10).

Regra prática: use `twinx()` apenas quando a comparação de unidades diferentes for genuinamente o ponto do gráfico (ex: "o Ibovespa reage à SELIC?") — e, quando usar, deixe explícito nos rótulos e cores qual linha

pertence a qual eixo (como no código acima, usando a mesma cor no eixo e na linha), para que o leitor nunca precise adivinhar.

3.3. Subplots: a alternativa mais honesta a dois eixos Y

Uma alternativa que evita o problema de escalas arbitrárias é simplesmente desenhar **dois gráficos separados**, um abaixo do outro, compartilhando o eixo X (o tempo):

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

ax1.plot(ipca.index, ipca["IPCA"], color="crimson")
ax1.set_title("IPCA")
ax1.grid(True, alpha=0.3)

ax2.plot(selic.index, selic["SELIC"], color="navy")
ax2.set_title("SELIC")
ax2.grid(True, alpha=0.3)

fig.tight_layout()
```

`sharex=True` faz os dois `Axes` compartilharem literalmente o mesmo eixo X (mesmos limites, mesmos ticks) — o que significa que, ao olhar verticalmente para um mês específico, o leitor vê o valor correspondente em ambos os gráficos, sem que nenhuma escala precise ser "encaixada" artificialmente na outra. A desvantagem é que fica mais difícil comparar **a forma exata** das duas curvas lado a lado (elas não estão sobrepostas) — um trade-off inerente entre "sobrepor com risco de distorção de escala" e "separar, perdendo a sobreposição direta".

Situação	Recomendação
Mesma unidade, magnitude parecida	Mesmo eixo Y, cores diferentes
Unidades muito diferentes, comparação de **forma/tendência** é o ponto	Subplots (`sharex=True`)
Unidades muito diferentes, mas é essencial ver ambas na mesma janela temporal sobrepostas	`twinx()`, com cuidado extra de rotulagem
Mais de duas séries com escalas diferentes	Preferir subplots — múltiplos eixos Y ficam ilegíveis rapidamente

3.4. Grade de subplots (mais de dois gráficos)

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
# axes é um array 2x2 de objetos Axes

axes[0, 0].plot(ipca.index, ipca["IPCA"], color="crimson")
axes[0, 0].set_title("IPCA")

axes[0, 1].plot(selic.index, selic["SELIC"], color="navy")
axes[0, 1].set_title("SELIC")

axes[1, 0].plot(cambio.index, cambio["USDBRL"], color="darkgreen")
axes[1, 0].set_title("Câmbio")

axes[1, 1].axis("off")  # deixa um "espaço" vazio, se sobrar um quadrante

fig.tight_layout()
```

Um jeito conveniente de simplificar o acesso aos eixos quando há muitos subplots é achatar o array com `.flatten()` e percorrê-lo com um `for` — reaproveitando exatamente o mesmo padrão de iteração da Apostila 1:

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
series = [("IPCA", ipca["IPCA"], "crimson"), ("SELIC", selic["SELIC"], "navy")]

for ax, (nome, dado, cor) in zip(axes.flatten(), series):
    ax.plot(dado.index, dado, color=cor)
    ax.set_title(nome)

fig.tight_layout()
```

3.5. Exemplo Resolvido — Escolhendo entre `twinx` e subplots

Enunciado: você quer comparar o Ibovespa (pontos, na casa de 100 mil) com a taxa SELIC (%, na casa de 10) entre 2020 e 2026, para argumentar que "o mercado de ações reagiu de forma inversa aos ciclos de aperto monetário". Justifique se `twinx()` ou subplots é a escolha mais honesta aqui, e implemente a opção escolhida.

Solução comentada:

Como o objetivo explícito é comparar a **direção** de movimento das duas séries no mesmo instante de tempo (subiu/desceu junto ou não), e não apenas suas tendências de longo prazo isoladas, `twinx()` é justificável aqui — mas com um cuidado adicional: **inverter o eixo do Ibovespa não seria honesto** só para "forçar" visualmente uma correlação negativa aparente; os limites de cada eixo devem ser definidos de forma neutra (por exemplo, começando ambos os eixos a partir de zero ou de um mínimo natural dos dados, nunca escolhidos "no olho" para maximizar a semelhança visual).

```
fig, ax1 = plt.subplots(figsize=(12, 5))

ax1.plot(ibov.index, ibov["IBOV"], color="darkgreen", linewidth=1.3, label="Ibovespa")
ax1.set_ylabel("Ibovespa (pontos)", color="darkgreen")
ax1.tick_params(axis="y", labelcolor="darkgreen")
ax1.set_ylim(bottom=0)  # eixo começa em zero — decisão neutra, não escolhida para
"encaixar" a outra série

ax2 = ax1.twinx()
ax2.plot(selic.index, selic["SELIC"], color="navy", linewidth=1.3, label="SELIC")
ax2.set_ylabel("SELIC (% a.a.)", color="navy")
ax2.tick_params(axis="y", labelcolor="navy")
ax2.set_ylim(bottom=0)

ax1.set_title("Ibovespa e SELIC — 2020 a 2026")
fig.tight_layout()
```

O ponto central da solução não é o código em si — é a justificativa: usar `twinx()` é uma decisão que deveria vir **depois** de decidir qual pergunta o gráfico responde, nunca antes. E, ao usá-lo, fixar os limites de cada eixo (`set_ylim`) de forma consistente com a natureza dos dados (aqui, ambos começando em zero, já que nem SELIC nem Ibovespa são negativos) remove um grau de liberdade que poderia ser usado para manipular a impressão visual do leitor.

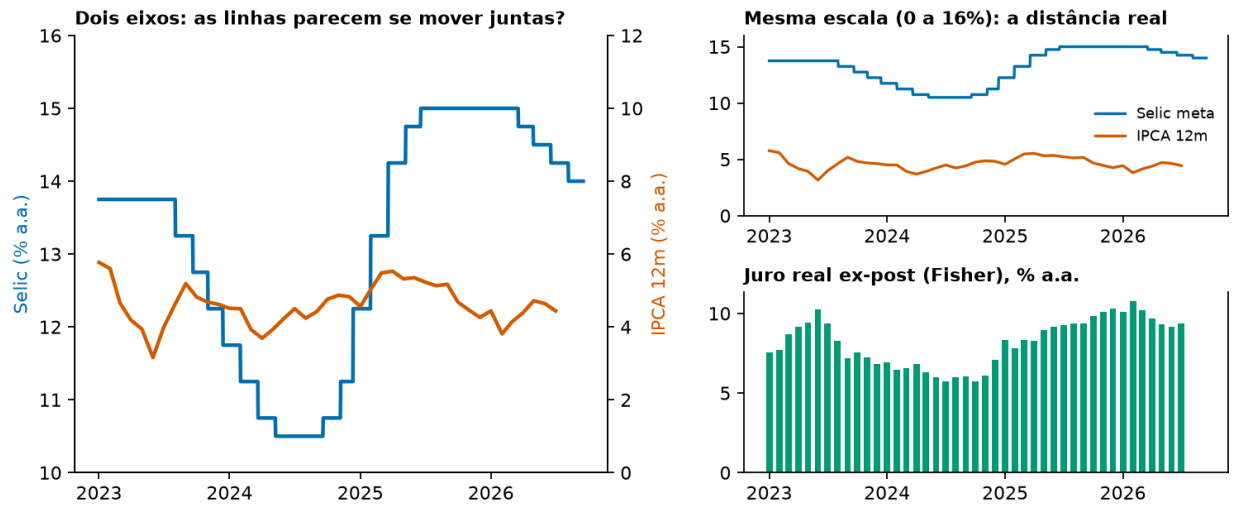
> **Aprofundando a teoria — dois eixos Y: por que a correlação visual é uma ilusão controlável**
> Com `twinx`, o autor escolhe **duas escalas independentes** — e, ao variar os limites dos eixos, consegue fazer duas séries "andarem juntas" ou "se descolarem" à vontade. O cruzamento das linhas, que o olho interpreta como evento relevante, é um **artefato** da escolha de escala. A alternativa honesta: painéis empilhados com o mesmo eixo de tempo (*subplots*), ou as duas séries na mesma escala quando têm a mesma unidade, ou ainda índices base 100. A figura usa a Selic e o IPCA em 12 meses — ambos em % ao ano, logo comparáveis na *mesma escala*:

```
fig = plt.figure(figsize=(9.5, 4.2))
gs = fig.add_gridspec(2, 2, width_ratios=[1.1, 1])
```

```
ax_dual = fig.add_subplot(gs[:, 0])
ax_dual.plot(sel3.index, sel3.values, color="#0072B2", linewidth=2)
ax_dual.set_ylabel("Selic (% a.a.)", color="#0072B2")
ax_b = ax_dual.twinx()
ax_b.plot(ip3.index, ip3.values, color="#D55E00", linewidth=2)
ax_b.set_ylabel("IPCA 12m (% a.a.)", color="#D55E00")
ax_dual.set_ylim(10, 16); ax_b.set_ylim(0, 12)                          # escolhas
arbitrárias que 'colam' as séries
ax_dual.set_title("Dois eixos: as linhas parecem se mover juntas?", loc="left",
fontsize=10, fontweight="bold")

ax_same = fig.add_subplot(gs[0, 1])
ax_same.plot(sel3.index, sel3.values, color="#0072B2", label="Selic meta")
ax_same.plot(ip3.index, ip3.values, color="#D55E00", label="IPCA 12m")
ax_same.set_ylim(0, 16); ax_same.legend(frameon=False, fontsize=8)
ax_same.set_title("Mesma escala (0 a 16%): a distância real", loc="left", fontsize=10,
fontweight="bold")
ax_gap = fig.add_subplot(gs[1, 1], sharex=ax_same)
juro_real = ((1 + sel3.resample("MS").last() / 100) / (1 + ip3 / 100) - 1).dropna()
ax_gap.bar(juro_real.index, juro_real.values * 100, width=20, color="#009E73")
ax_gap.set_title("Juro real ex-post (Fisher), % a.a.", loc="left", fontsize=10,
fontweight="bold")
for ax in (ax_dual, ax_b, ax_same, ax_gap):
    ax.spines[["top"]].set_visible(False);
ax.xaxis.set_major_locator(mdates.YearLocator());
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/dois_eixos_vs_subplots.png", dpi=160)
plt.close(fig)
print(f"Juro real ex-post: mínimo {juro_real.min() * 100:.1f}% | máximo {juro_real.max() *
100:.1f}% | último {juro_real.iloc[-1] * 100:.1f}%")
```

```
Juro real ex-post: mínimo 5.7% | máximo 10.8% | último 9.4%
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 3: À esquerda, o eixo duplo sugere uma relação que a escala própria de cada eixo cria; à direita, mesma escala e o diferencial (juro real) explícito.

Capítulo 4 — Gráfico de barras: comparação de categorias

4.1. Barra vs. linha: uma decisão sobre o tipo do eixo X

A regra mais importante para escolher entre linha e barra não é estética — é sobre **a natureza do eixo X**:

- Se o eixo X representa uma sequência **contínua e ordenada** (tempo, principalmente), uma **linha** é apropriada: ela comunica a ideia de continuidade e taxa de variação entre pontos adjacentes (Capítulo 2.1).
- Se o eixo X representa **categorias discretas sem ordem natural** (países, setores da economia, faixas de renda), uma **barra** é apropriada: cada categoria é isolada e comparável apenas pela altura (ou comprimento) da barra, sem nenhuma implicação de "continuidade" entre uma categoria e a vizinha.

```
dados = {
    "pais": ["Brasil", "Argentina", "Chile", "Colômbia", "Peru"],
    "pib": [2.2, -1.8, 3.1, 2.8, 2.5]
}
df = pd.DataFrame(dados).sort_values("pib")

fig, ax = plt.subplots(figsize=(10, 5))
cores = ["green" if v > 0 else "crimson" for v in df["pib"]]

ax.bar(df["pais"], df["pib"], color=cores, edgecolor="black", linewidth=0.5)
ax.axhline(0, color="black", linewidth=0.8)
ax.set_title("Crescimento do PIB — América do Sul", fontsize=14)
ax.set_ylabel("PIB (%)")
ax.grid(axis="y", alpha=0.3)
fig.tight_layout()
```

Ligar os cinco países acima com uma **linha** seria enganoso: sugeriria que "Argentina" está, de alguma forma, "entre" Brasil e Chile numa progressão contínua — o que não faz sentido, já que a ordem no eixo X (aqui, por valor de PIB, para facilitar a leitura) é arbitrária e não representa nenhuma grandeza contínua.

4.2. Comprimento, não área: por que a barra funciona

Voltando à hierarquia de Cleveland-McGill (Capítulo 2.1): um gráfico de barras codifica cada valor como o **comprimento** de um retângulo a partir de uma base comum (o zero) — a terceira codificação mais precisa da hierarquia, atrás apenas de posição ao longo de uma escala comum. É exatamente por isso que barras são preferíveis a gráficos de pizza (que codificam por **ângulo**, quinto lugar na hierarquia) ou bolhas proporcionais por área (sexto lugar) quando o objetivo é permitir comparações numéricas precisas entre categorias.

4.3. A regra do eixo zero em gráficos de barra

Esta é talvez a regra de honestidade visual mais importante em todo este material, e a mais frequentemente violada em gráficos de imprensa e redes sociais: **o eixo de um gráfico de barras deve sempre começar em zero.**

O motivo é direto: como a barra comunica o valor pelo **comprimento** do retângulo, cortar o eixo Y (por exemplo, começando em 80 em vez de 0) distorce a proporção visual entre as barras de um jeito que não corresponde à proporção real dos valores. Uma barra de PIB de "4%" que parece **quatro vezes maior** que uma barra de "1%" quando o eixo começa em zero pode parecer **quarenta vezes maior**, se o eixo for cortado para começar em 3,9% — mesmo que os números impressos ao lado das barras sejam honestos, a impressão visual instantânea (que é o que a maioria dos leitores realmente absorve) não é. Voltaremos a esse exemplo especificamente no Capítulo 10.

```
# ERRADO — distorce a comparação visual entre as barras
ax.bar(df["pais"], df["pib"])
ax.set_ylim(1.5, 3.5)   # corta o eixo, exagerando as diferenças

# CORRETO — o eixo Y de uma barra deve incluir o zero
```

```
ax.bar(df["pais"], df["pib"])
ax.set_ylim(0, df["pib"].max() * 1.1)  # margem de 10% acima do maior valor, mas começando
em 0
```

(Nota: **essa regra do eixo zero é específica de barras** — para gráficos de linha, cujo objetivo é mostrar a *variação* ao longo do tempo e não a comparação de comprimentos a partir de uma base, cortar o eixo Y é aceitável e às vezes até necessário para tornar visível uma variação sutil. A distinção importa e reaparece no Capítulo 10.)

4.4. Barras horizontais

Quando os rótulos das categorias são longos (nomes de países, setores, perguntas de uma pesquisa), barras **horizontais** evitam que o texto do eixo X fique cortado, girado ou ilegível:

```
fig, ax = plt.subplots(figsize=(8, 6))
df_ordenado = df.sort_values("pib")
ax.barh(df_ordenado["pais"], df_ordenado["pib"], color=cores)
ax.set_xlabel("PIB (%)")
ax.axvline(0, color="black", linewidth=0.8)
fig.tight_layout()
```

Uma boa prática adicional em barras horizontais: **ordenar as categorias pelo valor** (crescente ou decrescente), em vez de deixá-las em ordem alfabética — isso transforma o gráfico numa espécie de "ranking visual" instantâneo, o que quase sempre é mais útil economicamente do que a ordem alfabética dos nomes dos países.

4.5. Barras agrupadas e empilhadas

Para comparar duas ou mais séries por categoria (por exemplo, PIB em dois anos diferentes, por país), existem duas variações:

```
import numpy as np

paises = ["Brasil", "Chile", "Peru"]
pib_2024 = [3.1, 2.6, 3.3]
pib_2025 = [2.2, 3.1, 2.9]

x = np.arange(len(paises))
largura = 0.35

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(x - largura/2, pib_2024, largura, label="2024", color="steelblue")
ax.bar(x + largura/2, pib_2025, largura, label="2025", color="crimson")

ax.set_xticks(x)
ax.set_xticklabels(paises)
ax.legend()
ax.set_title("PIB por País — Barras Agrupadas")
fig.tight_layout()
```

Barras **empilhadas** (`bottom=...`) são apropriadas quando o objetivo é mostrar a **composição de um total** (por exemplo, decomposição do PIB por setor: agropecuária + indústria + serviços) — mas têm uma limitação importante: só a primeira "camada" da pilha começa em zero, então comparar visualmente o tamanho das camadas do meio é mais difícil (viola, de novo, a regra do Cleveland-McGill de comparar comprimentos a partir de uma base comum). Para esse caso, gráficos de barras lado a lado ou pequenos múltiplos (Capítulo 3.4) costumam comunicar melhor do que uma pilha única.

4.6. Exemplo Resolvido — Escolhendo e defendendo o tipo de gráfico

Enunciado: um colega sugere usar um gráfico de **linha** para comparar o PIB per capita de 8 países da América Latina em 2025 (um valor por país, sem dimensão temporal). Explique por que essa escolha é inadequada, proponha a alternativa correta, e implemente-a.

Solução comentada:

O eixo X, nesse caso, representaria **países** — uma categoria sem ordem natural nem continuidade. Uma linha ligando "Brasil" a "Chile" a "Peru" sugeriria implicitamente uma progressão ou relação de vizinhança entre os países que não existe (a não ser que a ordem do eixo X fosse, por exemplo, geográfica de norte a sul, o que ainda assim seria uma escolha arbitrária e não intrínseca ao dado). A alternativa correta é um gráfico de barras, ordenado pelo valor:

```
df = pd.DataFrame({
    "pais": ["Brasil", "Argentina", "Chile", "Colômbia", "Peru", "Uruguai", "Paraguai",
"Bolívia"],
    "pib_per_capita": [8917, 13700, 16500, 6800, 7500, 21700, 6100, 3700]
}).sort_values("pib_per_capita")

fig, ax = plt.subplots(figsize=(9, 6))
ax.barh(df["pais"], df["pib_per_capita"], color="steelblue", edgecolor="black",
linewidth=0.4)
ax.set_xlabel("PIB per capita (US$)")
ax.set_title("PIB per capita — América Latina (2025)")
fig.tight_layout()
```

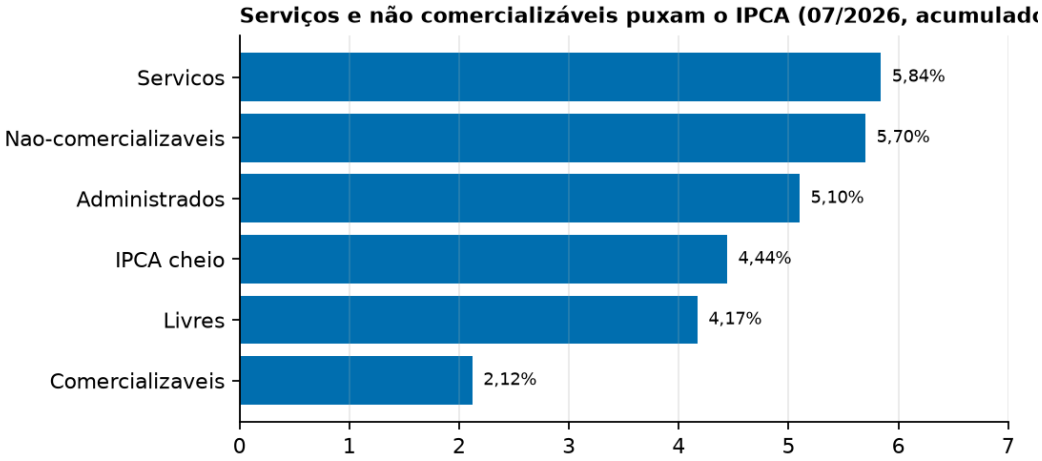
Barras horizontais, ordenadas por valor, comunicam diretamente um ranking — que é, na prática, a pergunta implícita por trás de "comparar 8 países": quem está melhor ou pior, e por quanto. Nenhuma linha é necessária, porque não há nenhuma continuidade real a representar entre um país e o próximo no eixo.

> **Aprofundando a teoria — barras codificam por comprimento: a hierarquia de Cleveland-McGill e a ordenação**
> Estudos de percepção (Cleveland e McGill, 1984) ordenaram as tarefas visuais por **precisão do julgamento**: o mais preciso é comparar *posições ao longo de uma escala comum* (pontos, barras com base comum), depois comprimentos, depois ângulos e inclinações, depois áreas, e por último cores e volumes. Barras funcionam porque usam comprimento a partir de uma **base comum** (zero); pizzas e bolhas usam ângulo e área, bem menos precisos. Duas regras práticas decorrem: **ordene as barras** (por valor, não alfabeticamente) e use **barras horizontais** quando os rótulos forem longos. A figura mostra o IPCA por grupo em julho de 2026:

```
ultimo = gr3.iloc[-1].sort_values()
fig, ax = plt.subplots(figsize=(7, 3.4))
cores = ["#D55E00" if v < 0 else "#0072B2" for v in ultimo.values]
ax.barh(ultimo.index, ultimo.values, color=cores)
ax.axvline(0, color="black", linewidth=0.8)                                  # base comum: o
zero está sempre visível
for y_, v_ in enumerate(ultimo.values):
    ax.text(v_ + (0.1 if v_ >= 0 else -0.1), y_, f"{v_:.2f}%".replace(".", ","),
va="center", ha="left" if v_ >= 0 else "right", fontsize=8)
ax.set_title(f"Serviços e não comercializáveis puxam o IPCA ({gr3.index[-1]:%m/%Y},
acumulado em 12 meses)", loc="left", fontsize=10, fontweight="bold")
ax.set_xlim(0, 7); ax.spines[["top", "right"]].set_visible(False); ax.grid(axis="x",
alpha=0.25)
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/barras_horizontais.png", dpi=160)
```

```
plt.close(fig)
print("Ordem das barras (menor -> maior):", list(ultimo.index))
```

```
Ordem das barras (menor -> maior): ['Comercializaveis', 'Livres', 'IPCA cheio',
'Administrados', 'Nao-comercializaveis', 'Servicos']
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 4: Barras horizontais, ordenadas e com rótulos diretos; a base comum (zero) é sempre visível.

Capítulo 5 — Dispersão (scatter): relação entre variáveis

5.1. O que um scatter revela que uma linha ou barra não revelam

Um gráfico de dispersão (*scatter plot*) representa cada observação como um ponto posicionado por **duas** variáveis simultaneamente (eixo X e eixo Y) — diferente de linha e barra, que tipicamente relacionam uma variável de interesse a uma variável de "índice" (tempo ou categoria). É a ferramenta certa quando a pergunta é **"como a variável A se relaciona com a variável B?"** — por exemplo, anos de escolaridade vs. PIB per capita, ou inflação vs. desemprego (a clássica Curva de Phillips).

```
df = pd.DataFrame({
    "educacao": [5.2, 6.1, 7.8, 8.3, 9.5, 10.2, 11.0, 12.1],
    "pib_per_capita": [3000, 4500, 5800, 7200, 8500, 10200, 12500, 15000]
})

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(df["educacao"], df["pib_per_capita"],
           color="steelblue", s=80, alpha=0.7, edgecolor="black")

ax.set_title("Educação vs PIB per capita")
ax.set_xlabel("Anos de estudo (média)")
ax.set_ylabel("PIB per capita (US$)")
fig.tight_layout()
```

`s=80` controla o tamanho dos pontos (em pontos², não pixels), e pode inclusive ser usado como uma **terceira dimensão** de dado, mapeando o tamanho do ponto a uma variável adicional (por exemplo, população do país) — uma técnica conhecida como *bubble chart*. Como área é uma codificação imprecisa (Capítulo 4.2), bubble charts devem ser usados com moderação, e nunca quando uma comparação numérica precisa é o objetivo principal.

5.2. Overplotting e transparência

Quando há muitas observações (milhares de municípios, por exemplo), pontos se sobrepõem e a nuvem se torna uma mancha ilegível — um problema chamado *overplotting*. A solução mais simples é reduzir a opacidade (`alpha`), de forma que regiões onde muitos pontos se sobrepõem fiquem visualmente mais "densas" (mais escuras), criando uma leitura aproximada de densidade sem nenhum cálculo estatístico adicional:

```
ax.scatter(df_grande["x"], df_grande["y"], alpha=0.05, s=10, color="steelblue")
```

Para volumes ainda maiores (dezenas de milhares de pontos), um histograma bidimensional (`ax.hexbin()` ou `ax.hist2d()`) costuma comunicar densidade melhor do que pontos semitransparentes, já que colore diretamente por contagem em vez de depender da sobreposição visual de transparências.

5.3. Linha de tendência — e o lembrete mais importante deste capítulo

Uma linha de tendência (regressão linear simples) frequentemente acompanha um scatter para sugerir a direção da relação entre as duas variáveis:

```
import numpy as np

z = np.polyfit(df["educacao"], df["pib_per_capita"], 1)  # ajusta uma reta (grau 1)
p = np.poly1d(z)  # transforma os coeficientes em uma função avaliável

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(df["educacao"], df["pib_per_capita"], color="steelblue", s=80, alpha=0.7,
edgecolor="black")
ax.plot(df["educacao"], p(df["educacao"]), "r--", alpha=0.7, label="Tendência linear")
ax.legend()
fig.tight_layout()
```

O lembrete que precisa acompanhar todo scatter com linha de tendência em um contexto econômico: correlação não implica causalidade. Um scatter que mostra educação e PIB per capita subindo juntos não prova que educação *causa* renda maior — pode haver uma terceira variável (qualidade institucional, por exemplo) influenciando ambas, ou a causalidade pode correr no sentido inverso (países mais ricos investem mais em educação). Um gráfico bem projetado pode e deve incluir essa ressalva explicitamente — em uma nota de rodapé, uma anotação, ou no texto que acompanha o gráfico (Capítulo 11) — precisamente porque a força visual de uma linha de tendência tende a implicar causalidade no olhar de quem lê rápido, mesmo quando o texto ao lado é cuidadoso.

5.4. Anotando pontos específicos

Quando um ou poucos pontos são especialmente relevantes para o argumento (por exemplo, destacar o Brasil dentro de uma nuvem de países), `ax.annotate()` permite rotular pontos individuais:

```
ax.annotate(
    "Brasil",
    xy=(7.8, 5800),           # coordenada do ponto (ancorada nos dados)
    xytext=(8.5, 4500),       # posição do texto (pode ser deslocada para não sobrepor o
ponto)
    fontsize=10,
    arrowprops=dict(arrowstyle="->", color="gray")
)
```

O parâmetro `xytext`, combinado com `arrowprops`, permite deslocar o texto para uma posição sem sobreposição, desenhando uma seta até o ponto real — útil quando vários pontos estão próximos e os rótulos, se colocados diretamente sobre os pontos, se sobreporiam entre si.

5.5. Exemplo Resolvido — Curva de Phillips: montando e interpretando o scatter

Enunciado: monte um scatter de desemprego (eixo X) contra inflação (eixo Y) para simular uma Curva de Phillips, adicione uma linha de tendência, e escreva a ressalva metodológica que deveria acompanhar esse gráfico caso ele fosse publicado como argumento de política econômica.

Solução comentada:

```
df = pd.DataFrame({
    "desemprego": [12.5, 11.8, 10.2, 9.5, 8.8, 8.1, 7.5, 7.0, 6.5],
    "inflacao":   [2.1, 2.8, 3.5, 4.2, 4.8, 5.5, 6.3, 7.1, 8.0]
})

z = np.polyfit(df["desemprego"], df["inflacao"], 1)
p = np.poly1d(z)

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(df["desemprego"], df["inflacao"], color="darkorange", s=90, alpha=0.8,
edgecolor="black")
ax.plot(df["desemprego"], p(df["desemprego"]), "k--", alpha=0.6, label="Tendência linear")

ax.set_title("Desemprego x Inflação (simulação)")
ax.set_xlabel("Taxa de desemprego (%)")
ax.set_ylabel("Inflação (%)")
ax.legend()
fig.tight_layout()
```

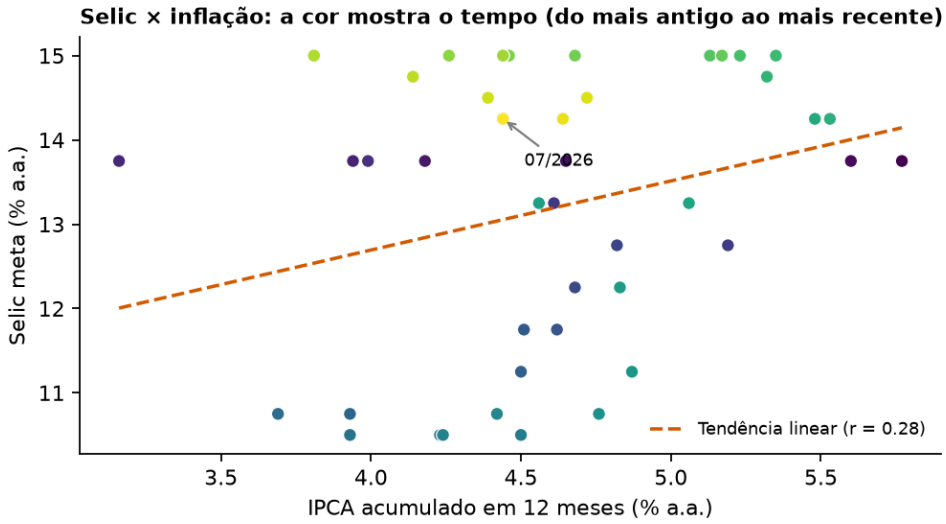
A ressalva necessária: esta relação negativa entre desemprego e inflação é a Curva de Phillips **na sua forma mais simples e mais antiga** (dados dos anos 1950-60) — economias modernas mostram que essa relação não é estável ao longo do tempo (a "Curva de Phillips aumentada por expectativas", de Friedman e Phelps, argumenta que ela se desloca conforme as expectativas de inflação mudam). Um gráfico como este, sem essa ressalva no texto que o acompanha, corre o risco de sugerir uma relação estrutural fixa onde, na verdade, existe uma relação condicional a um regime específico de política monetária e expectativas — um erro de interpretação clássico quando um scatter simples é lido sem o contexto teórico que o cerca.

> **Aprofundando a teoria — scatter, correlação e o gráfico de dispersão como instrumento exploratório**
> O scatter é a ferramenta natural para duas variáveis *contínuas*: mostra a **direção** da relação (sinal), sua **forma** (linear, curva, em grupos), a **dispersão** ao redor da tendência e os **outliers**. O coeficiente de correlação r resume só a parte linear — por isso se olha o gráfico *antes* de confiar no r (o "quarteto de Anscombe" mostra quatro conjuntos com o mesmo r e formas completamente diferentes). A linha de tendência é uma descrição, **não** uma prova de causalidade. Na figura, a Selic contra o IPCA em 12 meses, mês a mês, com ponto final anotado:

```
x_s, y_s = tay3["ipca12m"], tay3["selic"]
coef = np.polyfit(x_s, y_s, 1)
r_s = np.corrcoef(x_s, y_s)[0, 1]
fig, ax = plt.subplots(figsize=(6.4, 3.8))
ax.scatter(x_s, y_s, c=np.arange(len(x_s)), cmap="viridis", s=38, edgecolor="white",
linewidth=0.6)
xx = np.linspace(x_s.min(), x_s.max(), 50)
ax.plot(xx, np.polyval(coef, xx), color="#D55E00", linewidth=1.6, linestyle="--",
label=f"Tendência linear (r = {r_s:.2f})")
ax.annotate(f"{tay3.index[-1]:%m/%Y}", (x_s.iloc[-1], y_s.iloc[-1]), xytext=(10, -22),
textcoords="offset points",
            arrowprops=dict(arrowstyle="->", color="gray"), fontsize=8)
ax.set_xlabel("IPCA acumulado em 12 meses (% a.a.)"); ax.set_ylabel("Selic meta (% a.a.)")
ax.set_title("Selic × inflação: a cor mostra o tempo (do mais antigo ao mais recente)",
loc="left", fontsize=10, fontweight="bold")
```

```
ax.legend(frameon=False, fontsize=8); ax.spines[["top", "right"]].set_visible(False)
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/scatter_selic_ipca.png", dpi=160)
plt.close(fig)
print(f"Correlação (nível) Selic x IPCA 12m: {r_s:+.2f} | inclinação: {coef[0]:+.2f} p.p.
de Selic por p.p. de IPCA")
```

```
Correlação (nível) Selic x IPCA 12m: +0.28 | inclinação: +0.82 p.p. de Selic por p.p. de
IPCA
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 5: Scatter com cor para o tempo, linha de tendência e anotação do último ponto.

Atenção — erro comum
A correlação acima é calculada sobre **níveis** de duas séries que variam lentamente e com tendência. Em séries temporais, isso tende a superestimar a relação (correlação espúria). E a causalidade, aqui, é *bidirecional*: a Selic reage à inflação (regra de Taylor) e afeta a inflação futura, com defasagem. O gráfico mostra a associação; não responde "quem causa quem".

Capítulo 6 — Histograma: a forma da distribuição

6.1. O que um histograma mostra

Um histograma divide o intervalo de valores de uma variável em faixas (*bins*) e conta quantas observações caem em cada faixa, representando essa contagem como a altura de uma barra. Diferente do scatter (relação entre duas variáveis) e da linha (evolução no tempo), o histograma responde à pergunta **"como os valores de uma única variável se distribuem?"** — por exemplo: "a maioria dos meses teve IPCA entre 0,3% e 0,6%, ou os valores estão espalhados de forma mais uniforme?".

```
fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(ipca["IPCA"], bins=30, color="steelblue", edgecolor="black", alpha=0.7)
ax.axvline(ipca["IPCA"].mean(), color="red", linestyle="--", label=f"Média =
{ipca['IPCA'].mean():.2f}")
ax.axvline(ipca["IPCA"].median(), color="green", linestyle=":", label=f"Mediana =
{ipca['IPCA'].median():.2f}")
ax.legend()
```

```
ax.set_title("Distribuição do IPCA mensal (1995-2026)")
ax.set_xlabel("Variação mensal (%)")
ax.set_ylabel("Frequência")
fig.tight_layout()
```

6.2. A escolha do número de *bins* não é neutra

O parâmetro `bins` — quantas faixas dividir o intervalo de dados — tem um efeito surpreendentemente grande sobre a impressão visual da distribuição, e é uma das decisões mais subestimadas ao montar um histograma:

- **`bins` pequeno demais** (poucas faixas largas): esconde detalhes da distribuição — picos, assimetrias e multimodalidade (dois "grupos" distintos de valores) desaparecem, e tudo parece uma única distribuição suave.
- **`bins` grande demais** (muitas faixas estreitas): cada faixa recebe poucas observações, e o histograma vira um conjunto de barras irregulares, dominado por ruído amostral em vez de mostrar a forma real da distribuição.

Não existe um número "certo" universal, mas duas regras práticas ajudam a escolher um ponto de partida razoável, em vez de tentar valores ao acaso:

$$\text{Regra de Sturges:} \quad k = \lceil \log_2(n) + 1 \rceil$$

$$\text{Regra de Freedman-Diaconis: largura do bin} \quad = 2 \cdot \frac{\text{IQR}}{n^{1/3}}$$

onde n é o número de observações e IQR é o intervalo interquartil (visto em detalhe no Capítulo 7). A regra de Sturges funciona bem para amostras pequenas e distribuições aproximadamente normais; Freedman-Diaconis é mais robusta a outliers e é a regra padrão usada internamente por bibliotecas como `numpy` quando se passa `bins="fd"`:

```
ax.hist(ipca["IPCA"], bins="fd", color="steelblue", edgecolor="black")  # numpy escolhe
automaticamente
```

Na prática, o conselho mais importante não é decorar as fórmulas, mas **testar dois ou três valores de `bins` antes de decidir** — se a conclusão que você tira do gráfico muda drasticamente entre `bins=10` e `bins=50`, isso é um sinal de que a conclusão depende mais da escolha arbitrária de bins do que dos dados em si, e merece mais cautela antes de ser usada como argumento.

6.3. Assimetria (skewness) e o que ela significa economicamente

Muitas variáveis econômicas não seguem uma distribuição simétrica. A distribuição de renda, por exemplo, é classicamente **assimétrica à direita** (*right-skewed*): a maioria das observações está concentrada em valores baixos ou médios, com uma "cauda longa" de poucos valores muito altos puxando a média para cima. Um histograma revela isso visualmente de um jeito que apenas olhar "média" e "mediana" não revela sozinho:

Situação	O que o histograma mostra	Relação típica
Distribuição simétrica	Formato de "sino", cauda igual dos dois lados	Média ≈ Mediana
Assimétrica à direita (ex: renda, patrimônio)	Pico à esquerda, cauda longa à direita	Média > Mediana
Assimétrica à esquerda	Pico à direita, cauda longa à esquerda	Média < Mediana

É por isso que o Capítulo 6.1 traça **as duas** linhas verticais (média e mediana) sobre o histograma: a distância entre elas é, por si só, um diagnóstico visual rápido de assimetria, sem precisar calcular o coeficiente de assimetria formalmente.

6.4. Sobrepondo distribuições para comparação

Para comparar a distribuição de uma variável entre dois grupos (por exemplo, IPCA antes e depois de uma mudança de regime monetário), sobrepor dois histogramas com transparência é mais direto do que alterná-los em subplots separados, desde que o número de observações dos dois grupos seja comparável:

```
fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(ipca_pre_2016["IPCA"], bins=20, alpha=0.5, color="crimson", label="Até 2016",
density=True)
ax.hist(ipca_pos_2016["IPCA"], bins=20, alpha=0.5, color="steelblue", label="Após 2016",
density=True)
ax.legend()
ax.set_title("Distribuição do IPCA — Antes e Depois de 2016")
fig.tight_layout()
```

`density=True` normaliza cada histograma para que a área total sob as barras seja igual a 1, o que é essencial ao comparar grupos com números diferentes de observações — sem essa normalização, o grupo com mais dados sempre pareceria ter barras mais altas, mesmo que a **forma proporcional** da distribuição fosse idêntica à do outro grupo.

6.5. Exemplo Resolvido — Histograma e boxplot lado a lado

Enunciado: monte uma figura com dois `Axes` lado a lado (`subplots(1, 2)`): à esquerda, um histograma do IPCA mensal desde 1995; à direita, um boxplot da mesma série. Explique o que cada um revela que o outro não revela tão bem.

Solução comentada:

```
from bcb import sgs
import matplotlib.pyplot as plt

ipca = sgs.get({"IPCA": 433}, start="1995-01-01")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.hist(ipca["IPCA"], bins=30, color="steelblue", edgecolor="black")
ax1.set_title("Histograma")
ax1.set_xlabel("IPCA mensal (%)")

ax2.boxplot(ipca["IPCA"], vert=False)
ax2.set_title("Boxplot")
ax2.set_xlabel("IPCA mensal (%)")

fig.tight_layout()
fig.savefig("distribuicao_ipca.png", dpi=150)
```

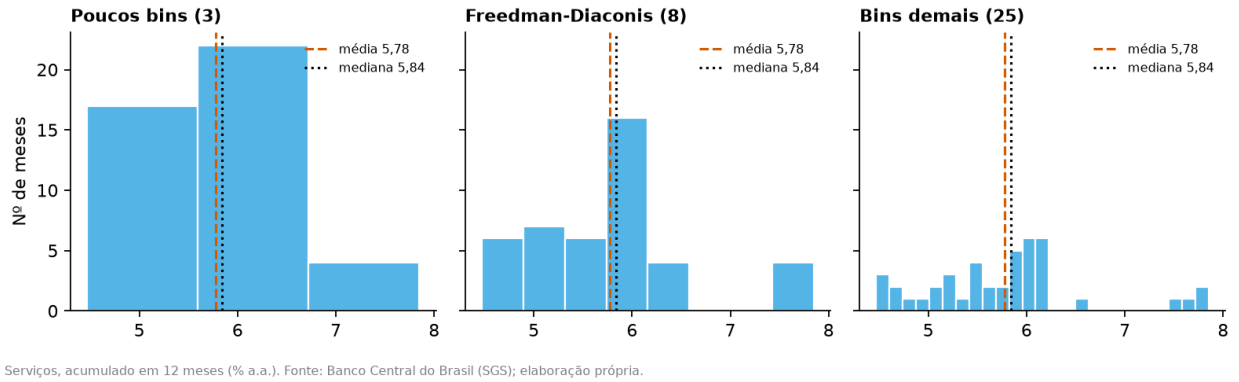
O histograma mostra a **forma completa** da distribuição — onde exatamente estão os picos, se há mais de um agrupamento de valores (bimodalidade, por exemplo, um IPCA "normal" e um IPCA de crise formando dois picos distintos), e o quão longa é a cauda. O boxplot, em contrapartida, resume a mesma distribuição em cinco números (mínimo, primeiro quartil, mediana, terceiro quartil, máximo, além dos outliers) e é muito mais compacto — o que o torna a ferramenta certa quando o objetivo é comparar **várias** distribuições lado a lado (o Capítulo 7 explora exatamente esse caso), mas pior quando o objetivo é entender os detalhes finos da forma de uma única distribuição.

> **Aprofundando a teoria — a escolha dos bins e a regra de Freedman-Diaconis**
> O histograma depende da largura das caixas (*bins*). Poucos bins escondem a forma; muitos criam ruído. Regras práticas: a de **Sturges** ($k \approx 1 + \log_2 n$, adequada para amostras pequenas e distribuições próximas da normal) e a de **Freedman-Diaconis** (largura $= 2 \cdot \frac{\text{IQR}}{n^{1/3}}$, robusta a outliers). A figura compara bins

demais e de menos para a mesma série e marca a média e a mediana (a distância entre elas denuncia assimetria):

```
serv = gr3["Servicos"]
largura_fd = 2 * (serv.quantile(0.75) - serv.quantile(0.25)) / len(serv) ** (1 / 3)
bins_fd = int(np.ceil((serv.max() - serv.min()) / largura_fd))
fig, axs = plt.subplots(1, 3, figsize=(10, 3.1), sharey=True)
for ax, n_bins, titulo in zip(axs, (3, bins_fd, 25), ("Poucos bins (3)", f"Freedman-
Diaconis ({bins_fd})", "Bins demais (25)")):
    ax.hist(serv, bins=n_bins, color="#56B4E9", edgecolor="white")
    ax.axvline(serv.mean(), color="#D55E00", linestyle="--", linewidth=1.4, label=f"média
{serv.mean():.2f}".replace(".", ","))
    ax.axvline(serv.median(), color="#000000", linestyle=":", linewidth=1.4,
label=f"mediana {serv.median():.2f}".replace(".", ","))
    ax.set_title(titulo, loc="left", fontsize=10, fontweight="bold"); ax.spines[["top",
"right"]].set_visible(False)
    ax.legend(frameon=False, fontsize=7)
axs[0].set_ylabel("Nº de meses")
fig.text(0.01, 0.01, "Serviços, acumulado em 12 meses (% a.a.). " + FONTE3, fontsize=7,
color="gray")
fig.tight_layout(rect=(0, 0.05, 1, 1))
fig.savefig("figuras03/histograma_bins.png", dpi=160)
plt.close(fig)
print(f"n = {len(serv)} | assimetria = {serv.skew():.2f} | bins pela regra de Freedman-
Diaconis: {bins_fd}")
```

```
n = 43 | assimetria = 0.88 | bins pela regra de Freedman-Diaconis: 8
```



Serviços, acumulado em 12 meses (% a.a.). Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 6: A mesma série em três escolhas de bins: a forma da distribuição só “aparece” no meio.

Capítulo 7 — Boxplot: resumo estatístico visual

7.1. Anatomia de um boxplot

Um boxplot (também chamado *box-and-whisker plot*, criado por John Tukey nos anos 1970) resume uma distribuição inteira usando cinco marcos estatísticos, sem exigir que o leitor veja cada observação individual:

$$\text{IQR} = Q_3 - Q_1$$

onde Q_1 é o primeiro quartil (25% dos dados abaixo dele) e Q_3 é o terceiro quartil (75% dos dados abaixo dele).

Elemento visual	O que representa
Linha dentro da caixa	Mediana (Q_2)

Elemento visual	O que representa
Base da caixa	Primeiro quartil (Q_1)
Topo da caixa	Terceiro quartil (Q_3)
Altura da caixa	Intervalo interquartil (IQR) — onde estão os 50% centrais dos dados
"Bigodes" (*whiskers*)	Tipicamente até $Q_1 - 1.5 \cdot \text{IQR}$ e $Q_3 + 1.5 \cdot \text{IQR}$
Pontos isolados além dos bigodes	Outliers — observações atípicas

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.boxplot(ipca["IPCA"], vert=True)
ax.set_title("Boxplot do IPCA mensal")
ax.set_ylabel("Variação (%)")
fig.tight_layout()
```

O multiplicador `1.5` que define o alcance dos bigodes é uma convenção (não uma lei estatística) proposta por Tukey — funciona bem como regra prática para sinalizar observações incomuns em distribuições aproximadamente simétricas, mas pode marcar como "outlier" observações perfeitamente normais em distribuições muito assimétricas (como renda). Vale sempre confirmar visualmente com um histograma (Capítulo 6) antes de tratar um ponto marcado como outlier no boxplot como um erro de dado.

7.2. Boxplot vs. histograma: quando usar cada um

Critério	Histograma	Boxplot
Mostra a forma completa (bimodalidade, cauda)	Sim	Não — resume em 5 números
Compacto o suficiente para comparar várias distribuições lado a lado	Difícil (sobreposição confusa)	Fácil — várias caixas lado a lado
Identifica outliers explicitamente	Indiretamente (barras isoladas)	Sim, marcados como pontos
Exige mais familiaridade estatística do leitor	Não (intuitivo)	Sim (requer entender quartis)

7.3. Comparando grupos com boxplot

O ponto forte do boxplot aparece quando há **várias** distribuições a comparar simultaneamente — por exemplo, a inflação de vários países no mesmo período:

```
df = pd.DataFrame({
    "Brasil": np.random.normal(4.5, 1.5, 100),
    "Chile": np.random.normal(3.0, 1.0, 100),
    "Argentina": np.random.normal(50, 20, 100),
})

fig, ax = plt.subplots(figsize=(8, 5))
df.boxplot(ax=ax)
ax.set_title("Distribuição da Inflação — Simulação")
ax.set_ylabel("Inflação (%)")
ax.grid(axis="y", alpha=0.3)
fig.tight_layout()
```

Nesse exemplo específico, note uma armadilha: a Argentina tem uma escala de inflação (~50%) muito maior que Brasil e Chile (~3-5%), então colocá-la no mesmo eixo Y "achata" visualmente as duas outras caixas — o mesmo problema de escala visto no Capítulo 3.2 com `twinx()`. Quando as magnitudes são tão diferentes, considerar subplots separados (um boxplot por país, com escalas independentes) ou uma transformação logarítmica do eixo Y é mais honesto do que forçar todas as caixas no mesmo eixo linear.

7.4. Exemplo Resolvido — Diagnosticando outliers antes de descartá-los

Enunciado: um boxplot do IPCA mensal desde 1995 marca vários pontos como outliers acima do bigode superior. Antes de simplesmente descartar essas observações como "erro", que investigação você faria, e como o histograma do Capítulo 6 ajudaria nessa investigação?

Solução comentada:

```
Q1 = ipca["IPCA"].quantile(0.25)
Q3 = ipca["IPCA"].quantile(0.75)
IQR = Q3 - Q1
limite_superior = Q3 + 1.5 * IQR

outliers = ipca[ipca["IPCA"] > limite_superior]
print(outliers)
```

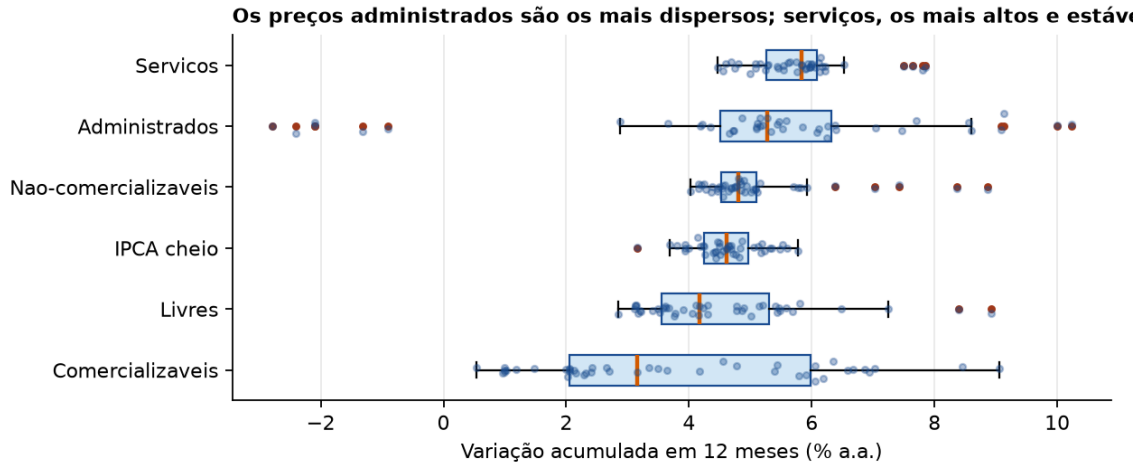
O raciocínio: antes de descartar esses pontos, é preciso perguntar **quando** eles ocorreram. Se `outliers.index` mostrar que todos os valores marcados caem em 2002-2003 (crise cambial) ou em 2015-2016 (recessão com forte depreciação do câmbio), isso não é ruído estatístico — é um regime econômico genuinamente diferente, e a "distribuição" do IPCA nesses períodos não deveria ser tratada como pertencente à mesma população dos períodos de estabilidade. Um outlier estatístico (definido apenas pela regra do IQR) e um dado errado (erro de digitação, falha de coleta) são coisas diferentes, e só o segundo deveria ser removido da análise; o primeiro, quando genuíno, geralmente é o dado **mais informativo** da série, não o menos.

> **Aprofundando a teoria — anatomia do boxplot e comparação entre grupos**
> O boxplot (Tukey) resume uma distribuição em cinco números: mínimo (do bigode), primeiro quartil, **mediana**, terceiro quartil e máximo do bigode; os pontos além de $1{,}5 \times \text{IQR}$ são marcados como *outliers*. Ele *comprime* a forma (não mostra bimodalidade) mas é imbatível para **comparar muitos grupos** lado a lado — por exemplo, os seis componentes do IPCA. Sobrepor os pontos individuais (*strip plot*) devolve a informação que a caixa esconde:

```
fig, ax = plt.subplots(figsize=(8, 3.6))
ordem = gr3.median().sort_values().index
ax.boxplot([gr3[c] for c in ordem], vert=False, tick_labels=ordem, widths=0.5,
patch_artist=True,
           boxprops=dict(facecolor="#d6eaf8", edgecolor="#1d4f91"),
medianprops=dict(color="#D55E00", linewidth=2),
           flierprops=dict(marker="o", markerfacecolor="#a33a1c", markersize=4,
markeredgecolor="none"))
rng_b = np.random.default_rng(1)
for i_, c in enumerate(ordem, start=1):
    ax.scatter(gr3[c], rng_b.normal(i_, 0.07, len(gr3)), s=10, color="#1d4f91", alpha=0.35,
zorder=3)
ax.set_xlabel("Variação acumulada em 12 meses (% a.a.)")
ax.set_title("Os preços administrados são os mais dispersos; serviços, os mais altos e
estáveis", loc="left", fontsize=10, fontweight="bold")
ax.spines[["top", "right"]].set_visible(False); ax.grid(axis="x", alpha=0.25)
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/boxplot_grupos.png", dpi=160)
plt.close(fig)
print("Mediana por grupo:", gr3.median().round(2).to_dict())
print("Amplitude interquartil:", (gr3.quantile(0.75) -
gr3.quantile(0.25)).round(2).to_dict())
```

```
Mediana por grupo: {'Servicos': 5.84, 'Administrados': 5.28, 'Livres': 4.17,
'Comercializaveis': 3.15, 'Nao-comercializaveis': 4.8, 'IPCA cheio': 4.61}
```

```
Amplitude interquartil: {'Servicos': 0.81, 'Administrados': 1.82, 'Livres': 1.75,
'Comercializaveis': 3.93, 'Nao-comercializaveis': 0.59, 'IPCA cheio': 0.71}
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 7: Boxplots horizontais ordenados pela mediana, com os pontos individuais sobrepostos.

Capítulo 8 — Heatmap e matrizes de correlação

8.1. Codificando valores por cor: sequencial vs. divergente

Um heatmap usa a cor de cada célula de uma grade para representar um valor numérico — útil quando os dados naturalmente formam uma matriz, como uma matriz de correlação entre várias variáveis econômicas. Escolher a **paleta de cores certa** depende da natureza dos dados:

- **Paleta sequencial** (ex: do branco/claro ao azul escuro): apropriada quando os valores vão de "baixo" a "alto" sem um ponto central neutro — por exemplo, PIB per capita por região, sempre positivo.
- **Paleta divergente** (ex: vermelho — branco — azul): apropriada quando existe um ponto neutro natural no meio da escala — como correlação, que varia de −1 a +1 com zero (ausência de correlação) como ponto central neutro.

Usar uma paleta sequencial para dados que têm um ponto neutro (como correlação) esconde a diferença qualitativa entre "correlação positiva" e "correlação negativa" — as duas pontas da escala parecem apenas "extremos", sem comunicar que representam relações de sinais opostos. É por isso que uma matriz de correlação quase sempre usa uma paleta divergente centrada em zero.

```
corr = df.corr(numeric_only=True)

fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)  # RdBu_r: vermelho (negativo) -
branco (zero) - azul (positivo)

ax.set_xticks(range(len(corr.columns)))
ax.set_yticks(range(len(corr.columns)))
ax.set_xticklabels(corr.columns, rotation=45, ha="right")
ax.set_yticklabels(corr.columns)

for i in range(len(corr.columns)):
    for j in range(len(corr.columns)):
        ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center", fontsize=9)
```

```
fig.colorbar(im, label="Correlação")
fig.tight_layout()
```

`vmin=-1, vmax=1` fixa explicitamente os limites da escala de cor — essencial em uma matriz de correlação, porque sem isso matplotlib normalizaria a cor automaticamente com base no menor e maior valor **presentes naquela matriz específica**, o que tornaria a intensidade da cor não comparável entre gráficos diferentes (uma correlação de 0.5 poderia parecer "forte" em uma matriz onde o valor máximo é 0.6, e "fraca" em outra onde o máximo é 0.95).

8.2. A armadilha do colormap "arco-íris" (jet)

Por muitos anos, o colormap padrão de diversas bibliotecas científicas (incluindo versões antigas de matplotlib) era o `jet` — uma sequência de cores passando por azul, ciano, verde, amarelo, laranja e vermelho. Esse colormap é hoje amplamente desaconselhado pela comunidade de visualização de dados, por dois motivos técnicos bem documentados:

1. **Não é perceptualmente uniforme**: a diferença de brilho entre cores adjacentes na escala não é constante — há trechos onde uma pequena mudança no valor gera uma mudança de cor dramática (a transição verde--amarelo, por exemplo), e outros onde uma mudança grande no valor mal se nota (dentro do trecho verde). Isso significa que o cérebro humano lê "importância" onde não necessariamente há, e ignora diferenças reais onde a cor muda pouco.
2. **Não funciona para daltônicos** (aproximadamente 8% dos homens têm alguma forma de daltonismo) e **não sobrevive à impressão em preto e branco**, já que as cores não têm uma progressão monotônica de brilho.

matplotlib substituiu `jet` por `viridis` como colormap padrão a partir da versão 2.0 (2017) — `viridis` foi desenhado especificamente para ser perceptualmente uniforme (a diferença visual entre duas cores é proporcional à diferença real nos dados) e permanecer legível tanto para daltônicos quanto em escala de cinza.

Tipo de dado	Colormap recomendado	Evitar
Sequencial (sempre positivo, sem ponto neutro)	`viridis`, `plasma`, `Blues`	`jet`, `rainbow`
Divergente (com ponto neutro, ex: correlação)	`RdBu_r`, `coolwarm`	`jet` (não tem ponto neutro claro)
Categórico (países, grupos sem ordem)	`tab10`, `Set2`	Qualquer colormap contínuo

8.3. Construindo um heatmap passo a passo

O processo, resumido: (1) calcular a matriz de valores (geralmente com `.corr()` do pandas, visto na Apostila 2); (2) desenhar a grade de cores com `ax.imshow()`; (3) rotular os eixos com os nomes das variáveis; (4) opcionalmente, escrever o valor numérico dentro de cada célula, para que o leitor não precise depender apenas da cor para ler valores precisos — uma boa prática geral: **cor sozinha raramente é suficiente para leitura precisa** (ela está no fim da hierarquia de Cleveland-McGill), então sempre que o espaço permitir, complementar a cor com o número exato é preferível.

8.4. Exemplo Resolvido — Corrigindo um heatmap mal projetado

Enunciado: o código abaixo gera um heatmap de correlação, mas comete três erros de projeto vistos neste capítulo. Identifique-os e corrija.

```
corr = df.corr(numeric_only=True)
fig, ax = plt.subplots()
im = ax.imshow(corr, cmap="jet")
fig.colorbar(im)
```

Solução comentada:

Os três erros: (1) `cmap="jet"` — não perceptualmente uniforme e não seguro para daltônicos (Seção 8.2); como os dados de correlação têm um ponto neutro natural (zero), o colormap correto é divergente, não arco-íris; (2) ausência de `vmin=-1, vmax=1` — sem fixar os limites, a escala de cor se ajusta ao intervalo específico daquela matriz, tornando comparações entre matrizes diferentes enganosas (Seção 8.1); (3) nenhum valor numérico escrito dentro das células — o leitor depende inteiramente da cor (fim da hierarquia perceptual) para julgar a intensidade de cada correlação, sem meio de confirmar o valor exato.

```
corr = df.corr(numeric_only=True)

fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)

ax.set_xticks(range(len(corr.columns)))
ax.set_yticks(range(len(corr.columns)))
ax.set_xticklabels(corr.columns, rotation=45, ha="right")
ax.set_yticklabels(corr.columns)

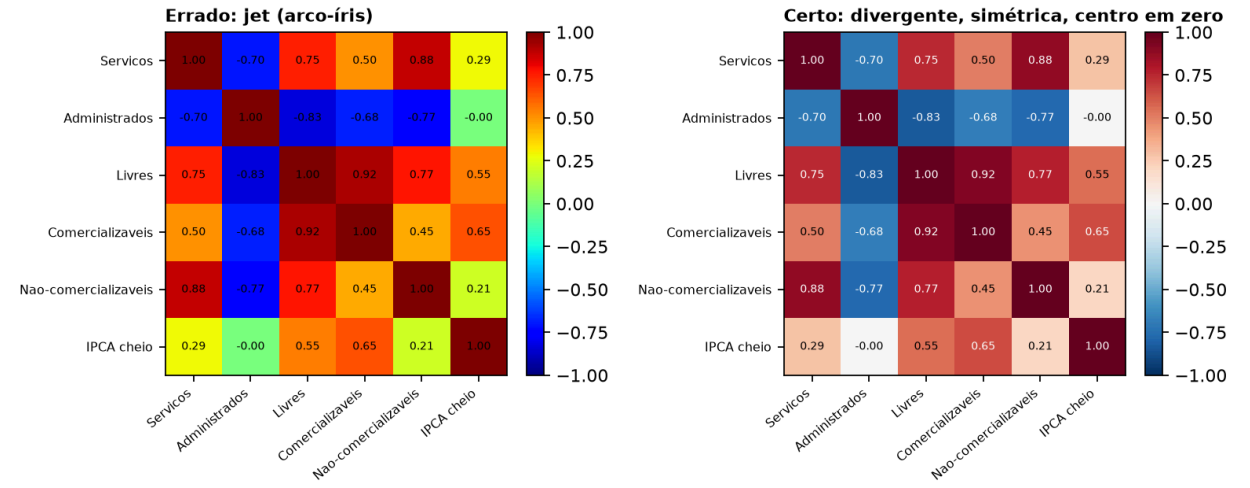
for i in range(len(corr.columns)):
    for j in range(len(corr.columns)):
        ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center", fontsize=9)

fig.colorbar(im, label="Correlação")
fig.tight_layout()
```

Aprofundando a teoria — mapa de calor: escala sequencial para magnitude, divergente para sinal
A cor de um heatmap codifica o *valor*. Duas famílias de escalas: a **sequencial** (um matiz que vai do claro ao escuro) serve para magnitudes que só crescem; a **divergente** (duas cores nas pontas, neutra no centro) serve quando há um *ponto de referência* com significado — como $r = 0$ numa correlação. Para uma matriz de correlação, a escala deve ser divergente, **simétrica** (-1 a $+1$) e com o centro no branco. O arco-íris (`jet`) é desaconselhado: não é perceptualmente uniforme (aparecem "faixas" falsas) e é ilegível para daltônicos. A figura contrasta a escolha errada e a certa:

```
corr = gr3.corr()
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4.2))
im1 = ax1.imshow(corr, cmap="jet", vmin=-1, vmax=1)
ax1.set_title("Errado: jet (arco-íris)", loc="left", fontsize=10, fontweight="bold")
im2 = ax2.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
ax2.set_title("Certo: divergente, simétrica, centro em zero", loc="left", fontsize=10,
fontweight="bold")
for ax, im in ((ax1, im1), (ax2, im2)):
    ax.set_xticks(range(len(corr))); ax.set_xticklabels(corr.columns, rotation=40,
ha="right", fontsize=7)
    ax.set_yticks(range(len(corr))); ax.set_yticklabels(corr.columns, fontsize=7)
    for i_ in range(len(corr)):
        for j_ in range(len(corr)):
            ax.text(j_, i_, f"{corr.iloc[i_, j_]:.2f}", ha="center", va="center",
fontsize=6.5,
                    color="white" if abs(corr.iloc[i_, j_]) > 0.6 and ax is ax2 else
"black")
    fig.colorbar(im, ax=ax, fraction=0.046)
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/heatmap_jet_vs_divergente.png", dpi=160)
plt.close(fig)
mais_alta = corr.where(np.triu(np.ones(corr.shape), 1).astype(bool)).stack().sort_values()
print(f"Maior correlação entre grupos: {mais_alta.index[-1]} = {mais_alta.iloc[-1]:.2f} |
menor: {mais_alta.index[0]} = {mais_alta.iloc[0]:.2f}")
```

```
Maior correlação entre grupos: ('Livres', 'Comercializaveis') = 0.92 | menor:
('Administrados', 'Livres') = -0.83
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 8: A mesma matriz de correlação: à esquerda o arco-íris, à direita a escala divergente centrada em zero.

Capítulo 9 — Teoria da percepção visual e escolha de cor

9.1. Retomando a hierarquia de Cleveland-McGill

Já usamos a hierarquia de Cleveland-McGill (Capítulo 2.1) para justificar escolhas específicas (linha, barra); vale agora consolidá-la como um princípio geral de projeto, aplicável a qualquer gráfico novo que você for desenhar:

$$\text{posição} > \text{comprimento} > \text{inclinação} > \text{ângulo} > \text{área} > \text{volume} > \text{cor}$$

Sempre que houver a opção de codificar um valor numérico usando uma codificação mais alta nessa hierarquia (posição, comprimento) em vez de uma mais baixa (área, cor), a mais alta deveria ser preferida — porque o olho humano consegue comparar duas posições ou dois comprimentos com muito mais precisão do que consegue comparar duas áreas ou duas tonalidades de cor. É por isso que um gráfico de barras (comprimento) é geralmente preferível a um gráfico de bolhas (área) ou a um mapa de calor (cor) para comparações numéricas exatas — a cor e a área são reservadas para quando a variável codificada é secundária ao argumento principal, ou quando há três ou mais dimensões a mostrar simultaneamente e uma delas precisa "ceder" para uma codificação menos precisa.

9.2. Três famílias de paleta de cor

Família	Quando usar	Exemplos
Categórica (*qualitative*)	Variáveis sem ordem — países, setores, partidos	`tab10`, `Set2`, `Paired`
Sequencial	Variáveis ordenadas, sempre no mesmo sentido (baixo→alto)	`viridis`, `Blues`, `plasma`
Divergente	Variáveis ordenadas com um ponto neutro central	`RdBu_r`, `coolwarm`, `PiYG`

Usar a família errada é um erro conceitual, não apenas estético: uma paleta categórica (cores sem relação de ordem entre si, como `tab10`) aplicada a uma variável contínua (por exemplo, colorir pontos de um scatter pela taxa de juros usando cores aleatórias e não relacionadas) faz o leitor perder completamente a noção de "qual cor é mais alta" — porque não existe uma progressão visual entre as cores de uma paleta categórica.

9.3. Daltonismo e paletas seguras

Aproximadamente um em cada doze homens (e uma em cada duzentas mulheres) tem alguma forma de daltonismo, sendo a deuteranopia (dificuldade em distinguir vermelho de verde) a mais comum. Um gráfico que codifica "positivo = verde, negativo = vermelho" — um padrão extremamente comum em gráficos econômicos (Capítulo 2.4) — é exatamente o par de cores mais problemático para essa parcela da audiência: as duas cores podem parecer indistinguíveis.

```
# Paleta segura para daltônicos (Paul Tol, amplamente adotada em publicações científicas)
cores_seguras = ["#0077BB", "#33BBEE", "#EE7733", "#CC3311", "#009988", "#BBBBBB"]
```

Duas mitigações práticas, que não exigem abandonar a intuição vermelho/verde por completo:

- **Redundância de codificação** (Seção 9.4 abaixo): usar também a posição (barras acima/abaixo do zero) ou um padrão de textura, não só a cor, para distinguir positivo de negativo.
- **Substituir por um par azul/laranja ou azul/vermelho-alaranjado**: mantém a leitura intuitiva de "duas categorias opostas" sem depender do contraste vermelho-verde especificamente.

9.4. Redundância de codificação: nunca dependa só da cor

Uma prática recomendada por padrão, e não apenas como mitigação de daltonismo: sempre que possível, codifique a mesma informação de **mais de uma forma simultaneamente** — cor **e** posição, cor **e** forma do marcador, cor **e** rótulo de texto. Isso não é redundância desperdiçada; é uma proteção contra qualquer limitação do canal de cor (impressão em preto e branco, daltonismo, um projetor mal calibrado numa apresentação).

O gráfico de barras do Capítulo 4.1 já faz isso corretamente: o PIB negativo é ao mesmo tempo vermelho **e** posicionado abaixo da linha de zero — um leitor que não distinguisse as cores ainda conseguiria identificar os países em recessão pela posição da barra.

9.5. Exemplo Resolvido — Auditando um gráfico quanto à acessibilidade de cor

Enunciado: um gráfico de dispersão usa `color="red"` para "países em recessão" e `color="green"` para "países em expansão", sem nenhuma outra distinção visual. Aponte o problema e proponha uma correção que resolva tanto a questão de daltonismo quanto o princípio de redundância de codificação.

Solução comentada:

O problema: a única forma de distinguir as duas categorias é a cor vermelho/verde — o par mais difícil para daltônicos com deuteranopia, e a informação se perde completamente se o gráfico for impresso em preto e branco. A correção soma uma segunda codificação (forma do marcador) à cor, criando redundância:

```
recessao = df[df["pib"] < 0]
expansao = df[df["pib"] >= 0]

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(recessao["x"], recessao["y"], color="#CC3311", marker="v", s=70,
label="Recessão", edgecolor="black")
ax.scatter(expansao["x"], expansao["y"], color="#0077BB", marker="^", s=70,
label="Expansão", edgecolor="black")
ax.legend()
fig.tight_layout()
```

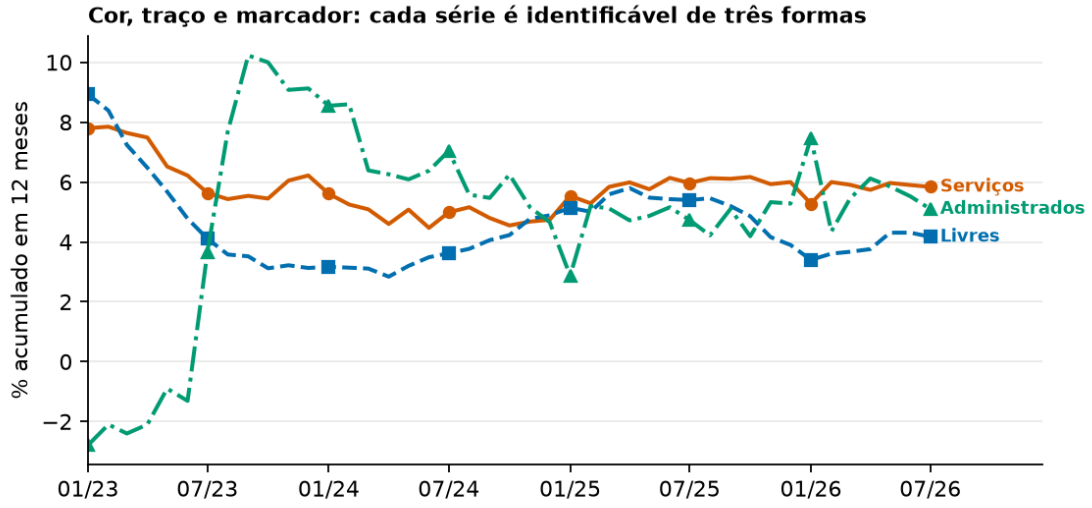
Agora, mesmo que as duas cores fossem indistinguíveis para um leitor específico, a direção do triângulo (v para baixo, ^ para cima) ainda comunica "recessão vs. expansão" de forma consistente com a semântica do próprio dado (para baixo = queda, para cima = crescimento) — um caso em que a redundância de codificação também reforça, e não apenas duplica, o significado.

> **Aprofundando a teoria — luminância, daltonismo e redundância: o que a paleta precisa garantir**
> Cerca de 8% dos homens têm alguma forma de daltonismo (mais comum, a confusão vermelho–verde). Uma paleta acessível precisa que as cores se distingam *por luminosidade* mesmo em escala de cinza, e que

> o gráfico **nunca dependa só da cor**: use também **estilo de linha, marcador e rótulos diretos**. A paleta de Okabe-Ito (usada aqui) foi desenhada para ser distinguível por quase todos. A figura codifica cada série com cor, traço e marcador — e o título e os rótulos diretos dispensam a legenda:

```
fig, ax = plt.subplots(figsize=(7.2, 3.6))
estilos = {"Servicos": ("#D55E00", "-", "o"), "Livres": ("#0072B2", "--", "s"),
"Administrados": ("#009E73", "-.", "^")}
for col, (cor, traco, marc) in estilos.items():
    ax.plot(gr3.index, gr3[col], color=cor, linestyle=traco, marker=marc, markevery=6,
linewidth=1.8, markersize=5)
    ax.text(gr3.index[-1] + pd.Timedelta(days=15), gr3[col].iloc[-1],
col.replace("Servicos", "Serviços"), color=cor, fontsize=8, va="center", fontweight="bold")
ax.set_xlim(gr3.index[0], gr3.index[-1] + pd.Timedelta(days=170))
ax.set_title("Cor, traço e marcador: cada série é identificável de três formas",
loc="left", fontsize=10, fontweight="bold")
ax.set_ylabel("% acumulado em 12 meses"); ax.spines[["top", "right"]].set_visible(False);
ax.grid(axis="y", alpha=0.25)
ax.xaxis.set_major_formatter(mdates.DateFormatter("%m/%y"))
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/redundancia_codificacao.png", dpi=160)
plt.close(fig)
print("Paleta Okabe-Ito:", OKABE)
```

```
Paleta Okabe-Ito: ['#E69F00', '#56B4E9', '#009E73', '#D55E00', '#0072B2', '#CC79A7',
'#000000']
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 9: Redundância de codificação: cor, estilo de traço, marcador e rótulo direto.

Capítulo 10 — Gráficos enganosos (“lying with charts”)

10.1. Por que este capítulo existe

Cada capítulo anterior mostrou como um gráfico bem projetado comunica informação de forma mais rápida e precisa do que uma tabela. A mesma força que torna um gráfico persuasivo — o fato de que o cérebro absorve uma impressão visual antes mesmo de ler os números — pode ser explorada, deliberadamente ou por descuido, para fazer um gráfico **honesto nos números, mas enganoso na impressão**. Como economista, você vai

encontrar (e, ocasionalmente, precisar identificar publicamente) gráficos desse tipo em relatórios, na imprensa e em redes sociais — este capítulo cataloga os padrões mais comuns.

10.2. Eixo Y cortado em gráfico de barras

Já visto no Capítulo 4.3, mas repetido aqui por ser o erro mais frequente: cortar o eixo Y de um gráfico de barras (não começar em zero) exagera visualmente as diferenças proporcionais entre categorias.

```
# Versão enganosa: sugere que o PIB "quase dobrou"
fig, ax = plt.subplots()
ax.bar(["2024", "2025"], [2.9, 3.1])
ax.set_ylim(2.8, 3.2)  # eixo cortado: a diferença de 0.2 p.p. parece enorme

# Versão honesta: a mesma diferença, no contexto correto
fig, ax = plt.subplots()
ax.bar(["2024", "2025"], [2.9, 3.1])
ax.set_ylim(0, 3.5)  # a diferença de 0.2 p.p. aparece na proporção real
```

Um `PIB de 2,9%` para `3,1%` é um aumento real de cerca de 7% em termos relativos — uma diferença moderada. Com o eixo cortado entre 2,8 e 3,2, a barra de 2025 ocupa visualmente **quatro vezes** a altura da de 2024, uma distorção grosseira que nenhum número na legenda desfaz completamente na primeira impressão do leitor.

10.3. Área e raio em vez de valor: o erro dos "círculos proporcionais"

Um erro comum, e frequentemente não intencional, em infográficos: representar um valor pelo **raio** de um círculo, quando a área (que é o que o olho realmente compara) cresce com o **quadrado** do raio. Se o PIB de um país é o dobro do de outro, e o círculo representando o segundo tem o dobro do raio do primeiro, a área do círculo maior é, na verdade, **quatro vezes** maior — uma distorção multiplicativa embutida na própria geometria do gráfico, não corrigida por nenhuma legenda.

$$A = \pi r^2 \implies \text{se} \;\; r_2 = 2r_1, \;\; \text{então} \;\; A_2 = 4A_1 \;\; (\text{não} \;\; 2A_1)$$

A correção correta, quando bolhas proporcionais são mesmo a escolha de design (por exemplo, para mostrar três dimensões simultaneamente, como em um gráfico de bolhas do Gapminder), é escalar a **área**, não o raio, proporcionalmente ao valor:

```
valores = np.array([10, 20, 40])
areas = valores  # escala linear na ÁREA, não no raio
raios = np.sqrt(areas / np.pi)  # deriva o raio a partir da área desejada

fig, ax = plt.subplots()
for i, (r, v) in enumerate(zip(raios, valores)):
    ax.add_patch(plt.Circle((i, 0), r, color="steelblue", alpha=0.6))
    ax.annotate(str(v), (i, 0), ha="center", va="center")
ax.set_xlim(-1, 3)
ax.set_ylim(-3, 3)
ax.set_aspect("equal")
```

Ainda assim, área permanece uma codificação de baixa precisão na hierarquia de Cleveland-McGill (Capítulo 9.1) — mesmo corrigida matematicamente, um leitor tem mais dificuldade em comparar duas áreas do que duas alturas de barra. A recomendação prática continua sendo: **prefira barras (comprimento) sempre que uma comparação numérica precisa for o objetivo**, e reserve bolhas para quando a terceira dimensão (o tamanho) é genuinamente secundária ao argumento.

10.4. Gráfico de pizza: ângulo é uma codificação fraca

Pelo mesmo motivo — ângulo está abaixo de comprimento na hierarquia perceptual —, gráficos de pizza dificultam comparar duas fatias que não sejam visualmente muito diferentes em tamanho. Comparar "qual fatia é maior, a de 23% ou a de 26%?" num gráfico de pizza exige um esforço visual real; a mesma comparação num gráfico de barras é imediata. Pizza também piora rapidamente com mais de 4-5 categorias, e piora ainda mais em

variações "3D" (que distorcem ainda mais o ângulo aparente das fatias por causa da perspectiva). Regra prática amplamente aceita na literatura de visualização de dados: **prefira barras a pizza quase sempre**; reserve pizza, no máximo, para duas ou três categorias com diferenças grandes e óbvias, onde a precisão da comparação importa menos que a ideia geral de "parte de um todo".

10.5. Eixos duplos manipulados

Retomando o Capítulo 3.2: como os limites de cada eixo Y em um gráfico `twinx()` são uma escolha livre de quem desenha o gráfico, é possível — inclusive sem intenção deliberada de enganar — escolher limites que fazem duas séries sem relação real parecerem se mover junto, simplesmente esticando ou comprimindo uma das escalas até que os picos e vales coincidam visualmente.

```
# Mesmo dado, dois gráficos com impressões completamente diferentes

# Versão 1: escalas "neutras" (cada eixo cobre o intervalo natural dos próprios dados)
ax2.set_ylim(serie_b.min() * 0.9, serie_b.max() * 1.1)

# Versão 2: escala escolhida especificamente para alinhar visualmente os picos com a série
A
ax2.set_ylim(valor_arbitrario_1, valor_arbitrario_2)  # "encaixado" a olho para parecer
correlacionado
```

Uma defesa prática contra essa armadilha, tanto para quem desenha quanto para quem avalia um gráfico de terceiros: perguntar explicitamente **"por que os limites do eixo são exatamente esses números, e quem os escolheu?"**. Se a resposta for "para que as duas curvas parecessem se sobrepor", isso é um sinal de alerta — os limites de um eixo deveriam derivar do intervalo natural dos próprios dados (mínimo/máximo, ou zero, conforme apropriado), não de uma tentativa de "encaixar" visualmente duas séries.

10.6. Escala logarítmica sem aviso

Uma escala logarítmica no eixo Y é uma ferramenta legítima e às vezes necessária (por exemplo, para comparar taxas de crescimento percentual ao longo de décadas, onde uma escala linear esconderia completamente as variações do início da série perto de valores pequenos). O problema não é usar escala log — é usá-la **sem indicar isso claramente** ao leitor, porque uma reta em escala logarítmica representa **crescimento exponencial constante**, não crescimento linear — uma leitura completamente diferente da intuição padrão de "reta = crescimento constante em unidades absolutas".

```
fig, ax = plt.subplots()
ax.plot(anos, pib_nominal)
ax.set_yscale("log")  # crucial indicar isso no título ou rótulo do eixo!
ax.set_ylabel("PIB nominal (R$, escala logarítmica)")  # o rótulo PRECISA dizer "log"
```

Um leitor que não perceber a escala logarítmica (porque o rótulo do eixo não avisou) pode interpretar uma reta como "o PIB cresceu a um ritmo constante em reais" quando, na verdade, ela significa "o PIB cresceu a uma **taxa percentual** aproximadamente constante" — duas afirmações econômicas muito diferentes.

10.7. Checklist de honestidade visual

Antes de considerar um gráfico econômico pronto para publicação, vale revisar:

Pergunta	Por quê
O eixo de um gráfico de barras começa em zero?	Evita exagerar diferenças de comprimento (Seção 10.2)
Se há dois eixos Y, seus limites derivam dos dados ou foram "encaixados" a olho?	Evita sugerir correlação espúria (Seção 10.5)
Se há uma escala logarítmica, isso está indicado no rótulo do eixo?	Evita leitura errada de "crescimento constante" (Seção 10.6)
Círculos/bolhas escalam por área, não por raio?	Evita exagero geométrico (Seção 10.3)
A cor é a única forma de distinguir categorias importantes?	Considere redundância de codificação (Capítulo 9.4)

Pergunta	Por quê
O título sugere uma conclusão que os dados realmente sustentam?	Um título pode ser tão enganoso quanto um eixo cortado

10.8. Exemplo Resolvido — Auditoria de um gráfico suspeito

Enunciado: um relatório mostra um gráfico de barras com o título "Explosão do desemprego em 2025!", eixo Y indo de 7% a 8%, comparando desemprego de 7,2% (2024) contra 7,6% (2025). Avalie o gráfico usando o checklist da Seção 10.7 e reconstrua uma versão honesta.

Solução comentada:

Dois problemas simultâneos: (1) o eixo Y cortado (de 7% a 8%, não de 0% a algo acima de 7,6%) faz uma diferença real de 0,4 ponto percentual — um aumento relativo de cerca de 5,6% — parecer uma barra quase 4 vezes maior; (2) o título ("Explosão") já entrega uma interpretação carregada antes mesmo de o leitor examinar os números, reforçando a impressão distorcida criada pelo eixo cortado.

```
# Reconstrução honesta
fig, ax = plt.subplots(figsize=(7, 5))
ax.bar(["2024", "2025"], [7.2, 7.6], color="steelblue", edgecolor="black")
ax.set_ylim(0, 10)
ax.set_title("Taxa de Desemprego — Leve Alta em 2025", fontsize=13)
ax.set_ylabel("Taxa de desemprego (%)")
for i, v in enumerate([7.2, 7.6]):
    ax.text(i, v + 0.15, f"{v}%", ha="center")
fig.tight_layout()
```

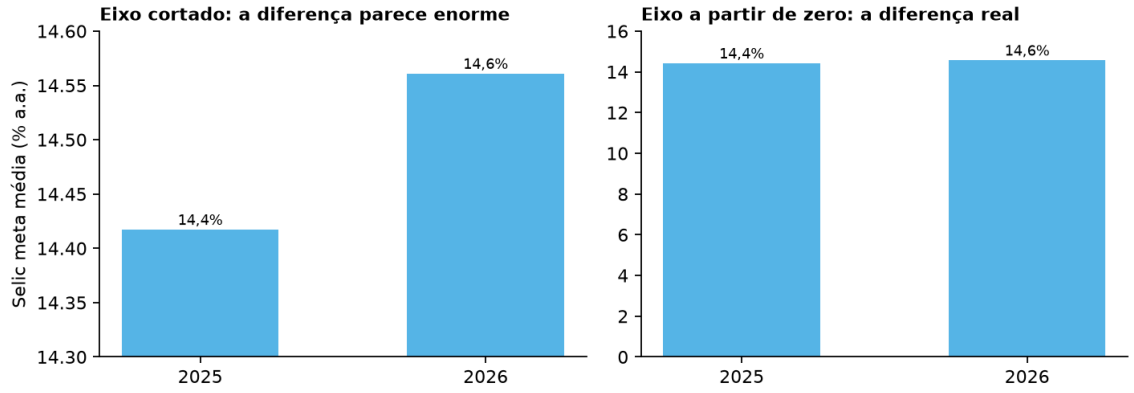
Com o eixo começando em zero e um título descritivo (não avaliativo), a mesma informação numérica passa a comunicar corretamente a magnitude real da mudança — uma alta moderada, não uma "explosão". Nenhum número foi alterado entre as duas versões; apenas as escolhas de eixo e de título, que são exatamente os pontos de manipulação mais comuns e mais difíceis de perceber a olho nu sem um checklist como este.

> **Aprofundando a teoria — eixo cortado: a inflação visual de uma diferença**
> Em gráficos de **barras**, o comprimento codifica o valor; cortar o eixo (começar em 14,3, em vez de 0) faz o *comprimento* deixar de ser proporcional ao valor: uma diferença de 1% pode parecer mais que o dobro. O "fator de mentira" (Tufte) é a razão entre o tamanho do efeito *no gráfico* e o efeito *nos dados* o ideal é 1. Em *linhas*, cortar o eixo é aceitável (a posição é o que importa), desde que o corte seja evidente. Veja a Selic média de 2025 e de 2026 nos dois desenhos:

```
anual = sel3.groupby(sel3.index.year).mean().loc[2025:2026]
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3.4))
for ax, (limites, titulo) in zip((ax1, ax2), (((14.3, 14.6), "Eixo cortado: a diferença
parece enorme"), ((0, 16), "Eixo a partir de zero: a diferença real"))):
    ax.bar(anual.index.astype(str), anual.values, color="#56B4E9", width=0.55)
    ax.set_ylim(*limites); ax.set_title(titulo, loc="left", fontsize=10, fontweight="bold")
    for x_, v_ in zip(anual.index.astype(str), anual.values):
        ax.text(x_, v_ + (limites[1] - limites[0]) * 0.015, f"{v_:.1f}%".replace(".", ","),
ha="center", fontsize=8)
    ax.spines[["top", "right"]].set_visible(False)
ax1.set_ylabel("Selic meta média (% a.a.)")
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/eixo_cortado.png", dpi=160)
plt.close(fig)
efeito_dados = (anual.iloc[-1] / anual.iloc[0] - 1) * 100
altura1, altura2 = anual.iloc[0] - 14.3, anual.iloc[-1] - 14.3                    #
alturas visíveis no gráfico cortado
efeito_visual = (altura2 / altura1 - 1) * 100
```

```
print(f"Selic média: {anual.round(2).to_dict()} (2026: jan a set)")
print(f"Diferença real: {efeito_dados:+.1f}% | diferença que o gráfico cortado sugere:
{efeito_visual:+.0f}% | fator de mentira: {efeito_visual / efeito_dados:.0f}")
```

```
Selic média: {2025: 14.42, 2026: 14.56} (2026: jan a set)
Diferença real: +1.0% | diferença que o gráfico cortado sugere: +123% | fator de mentira:
123
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 10: As mesmas três barras: o corte do eixo multiplica visualmente uma diferença pequena.

Capítulo 11 — Anotações, texto e storytelling com dados econômicos

11.1. Títulos que contam uma história

Um título genérico ("IPCA 2015-2026") apenas identifica o gráfico; um título descritivo entrega já a interpretação que os dados sustentam ("IPCA: da Crise de 2015-16 à Retomada de 2021", usado no Exemplo Resolvido do Capítulo 2). Essa prática — comum em veículos de dados como o *Financial Times* e o *The Economist* — não é uma forma de "viés" no gráfico, desde que a afirmação do título seja **de fato sustentada pelos dados** (diferente do título carregado do Exemplo Resolvido do Capítulo 10, que exagerava uma variação pequena). A diferença entre um bom título descritivo e um título enganoso não é o quão "forte" a linguagem é — é se a força da linguagem é proporcional à força real do padrão nos dados.

11.2. Anotando eventos: `axvline` e `annotate` juntos

Séries econômicas de longo prazo quase sempre atravessam eventos que explicam quebras estruturais — crises, mudanças de regime cambial, pandemias. Marcar esses eventos diretamente no gráfico poupa o leitor de precisar saber a cronologia de cor:

```
fig, ax = plt.subplots(figsize=(13, 5))
ax.plot(ipca.index, ipca["IPCA"], color="crimson", linewidth=1.2)

eventos = {
    "2016-01-01": "Teto de gastos\n(EC 95)",
    "2020-03-01": "Início da\npandemia",
}

for data, texto in eventos.items():
    ax.axvline(pd.Timestamp(data), color="gray", linestyle="--", linewidth=0.8)
    ax.annotate(texto, xy=(pd.Timestamp(data), ax.get_ylim()[1] * 0.9),
                fontsize=8, ha="center", color="dimgray")
```

```
ax.set_title("IPCA Mensal com Marcos Relevantes")
fig.tight_layout()
```

O cuidado ao anotar eventos: escolher **poucos e genuinamente relevantes** para o argumento do gráfico. Um gráfico com dez linhas verticais e dez rótulos de evento se torna tão poluído visualmente quanto um sem nenhuma anotação — o objetivo é guiar a leitura, não substituir o texto que acompanha o gráfico por uma lista de efemérides.

11.3. Créditos de fonte: uma prática de rigor, não de estilo

Todo gráfico econômico publicado deveria indicar, de forma discreta mas visível, a fonte dos dados e, quando relevante, a data de extração — porque séries econômicas frequentemente passam por revisões (o PIB de um trimestre pode ser revisado meses depois) e a proveniência do dado é parte da informação que o gráfico carrega:

```
fig.text(0.99, 0.01, "Fonte: BCB (SGS 433) — dados extraídos em 28/07/2026",
         ha="right", fontsize=8, color="gray")
```

11.4. Exemplo Resolvido — Transformando um gráfico "cru" em um gráfico que conta uma história

Enunciado: dado o gráfico simples de câmbio abaixo (sem título descritivo, sem anotações, sem fonte), reescreva-o para comunicar a história "o real se desvalorizou fortemente durante a crise fiscal de 2015, recuperou-se parcialmente, e voltou a depreciar durante a pandemia".

```
fig, ax = plt.subplots()
ax.plot(cambio.index, cambio["USDBRL"])
```

Solução comentada:

```
fig, ax = plt.subplots(figsize=(13, 5))
ax.plot(cambio.index, cambio["USDBRL"], color="darkgreen", linewidth=1.3)

eventos = {"2015-09-01": "Rebaixamento\nde rating", "2020-03-01": "Início da\npandemia"}
for data, texto in eventos.items():
    ax.axvline(pd.Timestamp(data), color="gray", linestyle="--", linewidth=0.8)
    ax.annotate(texto, xy=(pd.Timestamp(data), cambio["USDBRL"].max() * 0.95),
                fontsize=8, ha="center", color="dimgray")

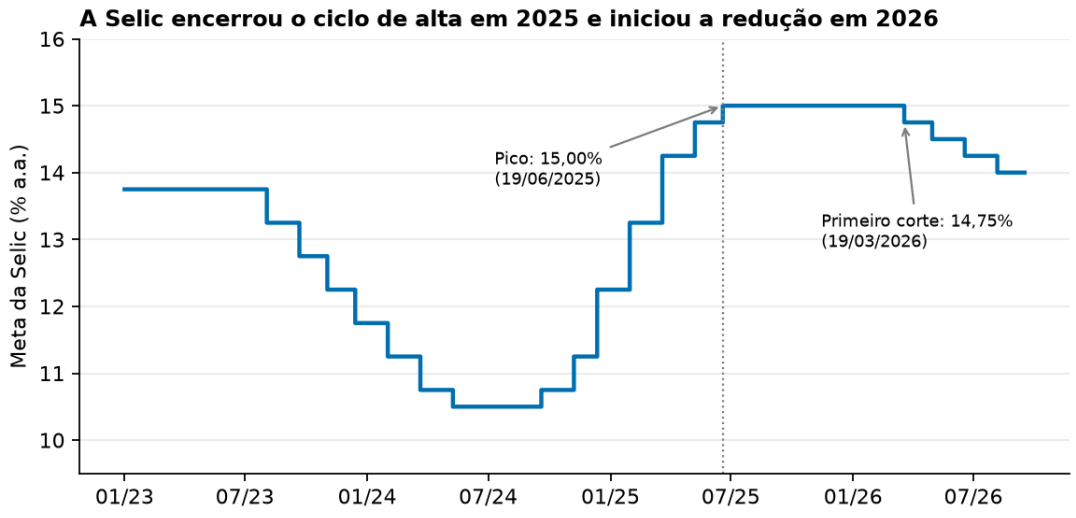
ax.set_title("Câmbio USD/BRL: Duas Depreciações Marcantes (2015 e 2020)", fontsize=13,
fontweight="bold")
ax.set_ylabel("R$ por US$")
ax.grid(True, alpha=0.25)
fig.text(0.99, 0.01, "Fonte: BCB (SGS 1) — dados extraídos em 28/07/2026", ha="right",
fontsize=8, color="gray")
fig.tight_layout()
```

Cada elemento adicionado (título específico, duas anotações de evento, fonte de dados) transforma o gráfico de um registro neutro de números em um argumento visual guiado — sem alterar um único valor da série original. Essa é a diferença central entre "plotar dados" (Capítulos 2 a 8) e "comunicar com dados" (este capítulo): a segunda etapa não substitui a primeira, ela a complementa.

> **Aprofundando a teoria — storytelling: título como conclusão, anotação como contexto**
> Um bom gráfico econômico conta *uma história*: o título afirma a conclusão ("a queda dos juros começou em março de 2026"), não descreve o eixo ("Selic ao longo do tempo"). As anotações (`axvline` + `annotate`) apontam os eventos que explicam o formato da curva, dispensando o leitor de procurar a causa no texto. A estrutura recomendada: **(1)** título-conclusão, **(2)** subtítulo com o recorte, **(3)** dados, **(4)** anotações em poucos pontos-chave, **(5)** fonte. A figura anota os dois marcos do ciclo da Selic nos dados:

```
mudancas = sel3[sel3.diff().fillna(0) != 0]
pico_data = sel3.idxmax()
primeiro_corte = mudancas[(mudancas.index > pico_data)].index[0]
fig, ax = plt.subplots(figsize=(7.4, 3.8))
ax.step(sel3.index, sel3.values, where="post", color="#0072B2", linewidth=2)
ax.axvline(pico_data, color="gray", linestyle=":", linewidth=1)
ax.annotate(f"Pico: {sel3.max():.2f}%".replace(".", ",") + f"\n({pico_data:%d/%m/%Y})",
(pico_data, sel3.max()), xytext=(-110, -38),
            textcoords="offset points", arrowprops=dict(arrowstyle="->", color="gray"),
fontsize=8)
ax.annotate(f"Primeiro corte: {sel3.loc[primeiro_corte]:.2f}%".replace(".", ",") +
f"\n({primeiro_corte:%d/%m/%Y})",
            (primeiro_corte, sel3.loc[primeiro_corte]), xytext=(-40, -60),
textcoords="offset points",
            arrowprops=dict(arrowstyle="->", color="gray"), fontsize=8)
ax.set_ylim(9.5, 16)
ax.set_title("A Selic encerrou o ciclo de alta em 2025 e iniciou a redução em 2026",
loc="left", fontsize=11, fontweight="bold")
ax.set_ylabel("Meta da Selic (% a.a.)"); ax.spines[["top", "right"]].set_visible(False);
ax.grid(axis="y", alpha=0.25)
ax.xaxis.set_major_formatter(mdates.DateFormatter("%m/%y"))
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/selic_storytelling.png", dpi=160)
plt.close(fig)
print(f"Pico em {pico_data:%d/%m/%Y} ({sel3.max()}%) | primeiro corte em {primeiro_corte:
%d/%m/%Y} (para {sel3.loc[primeiro_corte]}%)")
```

```
Pico em 19/06/2025 (15.0%) | primeiro corte em 19/03/2026 (para 14.75%)
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 11: Título que afirma a conclusão e duas anotações que explicam a curva (os valores vêm dos próprios dados).

Capítulo 12 — Personalização avançada e gráficos publication-ready

12.1. `rcParams`: configurando o estilo padrão uma única vez

Repetir `fontsize=12` em toda chamada de `set_title`, `set_xlabel` etc. em um script com vários gráficos é repetitivo e propenso a inconsistência. `plt.rcParams` permite configurar um padrão global, aplicado a todos os gráficos seguintes no mesmo script:

```
plt.rcParams.update({
    "font.size": 12,
    "axes.titlesize": 14,
    "axes.labelsize": 12,
    "legend.fontsize": 10,
    "figure.dpi": 100,
    "axes.grid": True,
    "grid.alpha": 0.3,
})
```

12.2. Removendo bordas desnecessárias (spines)

Por padrão, matplotlib desenha uma caixa completa ao redor de cada `Axes` (quatro bordas — *spines*). As bordas superior e direita raramente carregam informação (não há eixo Y à direita nem eixo X acima, na maioria dos gráficos), e removê-las é uma prática comum em gráficos de publicações como o *The Economist*, deixando o gráfico visualmente mais leve sem perder nenhuma informação:

```
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
```

12.3. Formatando eixos de data, milhar e percentual

O submódulo `matplotlib.ticker` controla precisamente como os valores dos eixos são formatados e espaçados — essencial para eixos de tempo (mostrar só o ano, não cada mês) e para eixos monetários (separador de milhar):

```
import matplotlib.ticker as ticker

# Eixo X de datas: um rótulo a cada 2 anos, mostrando só o ano
ax.xaxis.set_major_locator(ticker.MultipleLocator(2))
ax.xaxis.set_major_formatter(ticker.DateFormatter("%Y"))

# Eixo Y com separador de milhar
ax.yaxis.set_major_formatter(ticker.FuncFormatter(lambda x, _: f"{x:,.0f}"))

# Eixo Y como percentual (multiplica por 100 e adiciona %)
ax.yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1.0))
```

12.4. Exemplo Resolvido — O gráfico publication-ready completo

Enunciado: monte um único gráfico que combine as técnicas dos Capítulos 2, 3, 9, 11 e 12: IPCA e SELIC desde 2000, com preenchimento sob a curva do IPCA, spines removidas, grade sutil, legenda estilizada, e eixo X formatado por ano.

Solução comentada:

```
from bcb import sgs
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

ipca = sgs.get({"IPCA": 433}, start="2000-01-01")
```

```
selic = sgs.get({"SELIC": 11}, start="2000-01-01")

fig, ax = plt.subplots(figsize=(12, 5.5))

ax.fill_between(ipca.index, ipca["IPCA"], 0, alpha=0.1, color="crimson")
ax.plot(ipca.index, ipca["IPCA"], color="crimson", linewidth=1.2, label="IPCA")
ax.plot(selic.index, selic["SELIC"], color="navy", linewidth=1.2, label="SELIC")

ax.axhline(0, color="gray", linewidth=0.5)

ax.set_title("Brasil: Taxa SELIC e IPCA (2000-2026)", fontsize=14, fontweight="bold")
ax.set_ylabel("Percentual (%)")
ax.legend(frameon=True, fancybox=True, shadow=True)

ax.grid(True, alpha=0.3, linestyle=":")
ax.set_axisbelow(True)  # grade atrás dos dados, não sobre eles

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.xaxis.set_major_locator(ticker.MultipleLocator(2))
ax.xaxis.set_major_formatter(ticker.DateFormatter("%Y"))

fig.text(0.99, 0.01, "Fonte: BCB (SGS 433, 11)", ha="right", fontsize=8, color="gray")
fig.tight_layout()
fig.savefig("publication_ready.png", dpi=300, bbox_inches="tight")
plt.close(fig)
```

`ax.set_axisbelow(True)` é um detalhe fácil de esquecer: sem ele, a grade é desenhada **por cima** das linhas de dados (já que grades são adicionadas depois, por padrão), tornando a grade visualmente competitiva com a própria informação que deveria estar em primeiro plano. `fill_between(..., 0, alpha=0.1, ...)` preenche a área entre a curva do IPCA e o eixo zero com baixa opacidade — uma técnica comum para destacar visualmente qual das duas séries é a "protagonista" do gráfico, sem escurecer a leitura da segunda linha.

> **Aprofundando a teoria — de gráfico de exploração a gráfico de publicação: o checklist**
> Um gráfico para **você** (exploratório) pode ser rápido e feio; um gráfico para **publicar** precisa passar pelo checklist: título-conclusão; eixos com unidade; escala honesta; poucas cores com função; rótulos diretos; fonte e data; sem moldura desnecessária; fontes legíveis no tamanho final de impressão (8 a 10 pontos); resolução ou formato vetorial adequado. A figura final aplica os passos — `rcParams` para o estilo, remoção de bordas, formatação de data e eixo, rótulos diretos e fonte — sobre a Selic e a Selic implícita da regra de Taylor:

```
estilo = {"font.size": 9, "axes.spines.top": False, "axes.spines.right": False,
"axes.grid": True, "grid.alpha": 0.25,
          "axes.titleweight": "bold", "axes.titlelocation": "left"}
with plt.rc_context(estilo):
    fig, ax = plt.subplots(figsize=(7.4, 3.8))
    ax.plot(tay3.index, tay3["selic"], color="#0072B2", linewidth=2.2)
    ax.plot(tay3.index, tay3["taylor_selic"], color="#D55E00", linewidth=2.2)
    ax.fill_between(tay3.index, tay3["selic"], tay3["taylor_selic"], where=tay3["selic"] >=
tay3["taylor_selic"], color="#0072B2", alpha=0.12)
    ax.text(tay3.index[-1], tay3["selic"].iloc[-1] + 0.35, "Selic meta", color="#0072B2",
ha="right", fontweight="bold", fontsize=9)
    ax.text(tay3.index[-1], tay3["taylor_selic"].iloc[-1] - 0.75, "Regra de Taylor",
color="#D55E00", ha="right", fontweight="bold", fontsize=9)
    ax.set_title("A Selic ficou sistematicamente acima do que a regra de Taylor sugeria")
    ax.set_ylabel("% ao ano"); ax.set_ylim(7, 16)
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%m/%y"))
```

```
    fig.text(0.01, 0.01, "Taylor: r* = BCB (RPM), π* = meta CMN, π = IPCA 12m, sem hiato. "
+ FONTE3, fontsize=7, color="gray")
    fig.tight_layout(rect=(0, 0.04, 1, 1))
    fig.savefig("figuras03/publicacao_taylor.png", dpi=160)
    fig.savefig("figuras03/publicacao_taylor.pdf")
    plt.close(fig)
print(f"Gap médio (Selic - Taylor): {tay3['gap_pp'].mean():+.2f} p.p. | meses com Selic
acima da regra: {(tay3['gap_pp'] > 0).mean():.0%}")
```

```
Gap médio (Selic - Taylor): +3.04 p.p. | meses com Selic acima da regra: 100%
```

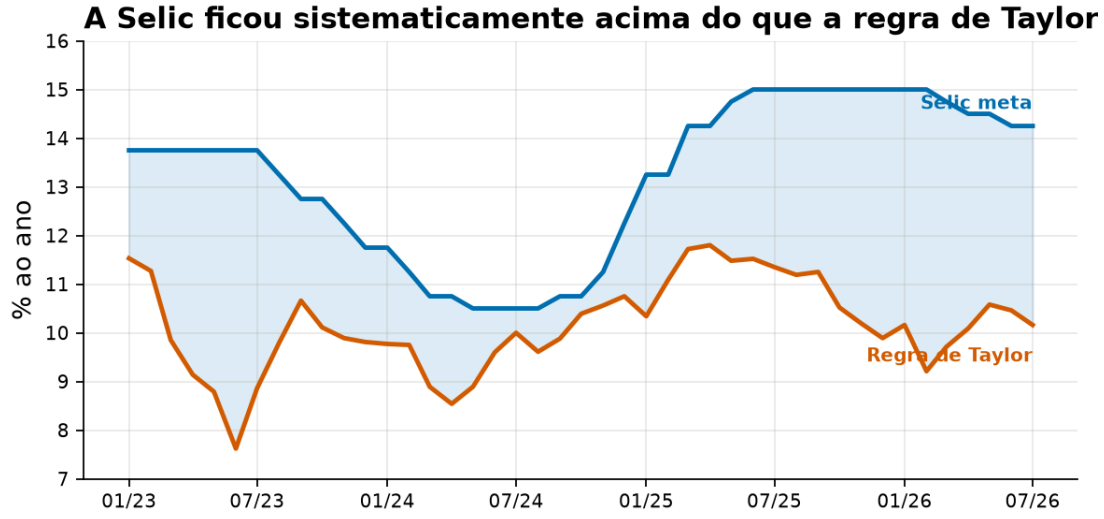


Figura 12: Gráfico “pronto para publicar”: estilo configurado uma vez (`rc_context`), rótulos diretos e fonte indicada. Um PDF vetorial foi gerado ao lado do PNG.

Capítulo 13 — seaborn: gráficos estatísticos de alto nível

13.1. A relação entre seaborn e matplotlib

seaborn é construído inteiramente em cima de matplotlib — toda figura gerada por `sns.algumacoisa()` é, por baixo, um objeto `Figure`/`Axes` de matplotlib, e pode ser customizada com os mesmos métodos (`ax.set_title()`, `ax.grid()`) vistos nos capítulos anteriores. O que seaborn adiciona é: (1) um tema visual padrão mais elaborado (grades suaves, paletas cuidadosamente escolhidas); (2) funções de alto nível que combinam várias etapas de matplotlib em uma única chamada (por exemplo, calcular e desenhar uma reta de regressão com intervalo de confiança, em uma linha); e (3) integração nativa com `DataFrames` do pandas, aceitando nomes de coluna diretamente como parâmetros.

```
import seaborn as sns

sns.set_theme(style="whitegrid")  # aplica o tema padrão a TODOS os gráficos seguintes no
script
```

13.2. Funções de eixo (*axes-level*) vs. de figura (*figure-level*)

seaborn distingue dois tipos de função, uma diferença que confunde muitos iniciantes:

- **Funções de eixo** (`sns.lineplot`, `sns.scatterplot`, `sns.boxplot`, `sns.heatmap`): desenham dentro de um `Axes` específico, aceitando o parâmetro `ax=` — funcionam exatamente como `ax.plot()` de matplotlib, e podem ser combinadas livremente em subplots feitos manualmente.

- **Funções de figura** (`sns.relplot`, `sns.catplot`, `sns.pairplot`, `sns.lmplot`): criam sua **própria** `Figure` internamente (não aceitam `ax=`), e são úteis quando o objetivo é gerar automaticamente uma grade de subplots dividida por uma variável categórica (por exemplo, um gráfico por país, automaticamente).

```
# Função de eixo: combina com o Axes que você já criou
fig, ax = plt.subplots(figsize=(12, 5))
sns.lineplot(data=ipca, x=ipca.index, y="IPCA", color="crimson", ax=ax)
sns.lineplot(data=selic, x=selic.index, y="SELIC", color="navy", ax=ax)
fig.tight_layout()

# Função de figura: cria a Figure e os Axes sozinha, dividindo por categoria
sns.relplot(data=df_paises, x="ano", y="pib", col="pais", kind="line", col_wrap=3)
```

13.3. Regressão com intervalo de confiança

Uma das conveniências mais úteis de seaborn é `sns.regplot`, que combina scatter, reta de regressão e uma faixa de incerteza (intervalo de confiança de 95%, por padrão, estimado por *bootstrap*) em uma única chamada — o que exigiria várias linhas manuais em matplotlib puro:

```
fig, ax = plt.subplots(figsize=(8, 6))
sns.regplot(data=df, x="educacao", y="pib_per_capita", ax=ax,
            scatter_kws={"alpha": 0.6, "color": "steelblue"},
            line_kws={"color": "crimson"})
ax.set_title("Educação vs PIB per capita (com IC 95%)")
fig.tight_layout()
```

A faixa sombreada ao redor da linha de tendência comunica visualmente a **incerteza** da estimativa — mais larga onde há poucos dados ou mais dispersão, mais estreita onde a relação é bem determinada. É uma forma visual do mesmo tipo de raciocínio estatístico (intervalo de confiança) que será formalizado na Apostila 4 (Econometria), aqui apresentado como um elemento gráfico antes de qualquer teoria de inferência.

13.4. `pairplot`: todas as correlações de uma vez

`sns.pairplot` gera automaticamente uma grade de scatter plots para cada par de variáveis numéricas de um `DataFrame`, com histogramas na diagonal — uma forma rápida de fazer uma primeira exploração visual de várias variáveis simultaneamente, antes de qualquer modelagem:

```
sns.pairplot(df[["pib", "inflacao", "desemprego", "populacao"]])
```

Essa é tipicamente uma das primeiras coisas a rodar ao receber uma base de dados nova com várias variáveis numéricas — antes mesmo de calcular uma matriz de correlação (Capítulo 8), o `pairplot` já revela visualmente se alguma relação é não linear (o que a correlação de Pearson, um número único, não capturaria bem) ou se há outliers óbvios que merecem investigação antes de qualquer modelo.

13.5. Boxplot com seaborn: menos código, mesmo resultado

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.boxplot(data=df, ax=ax, palette="Set2")
ax.set_title("Distribuição — seaborn")
fig.tight_layout()
```

13.6. Exemplo Resolvido — Recriando um gráfico matplotlib em seaborn e comparando o código

Enunciado: reescreva, usando `sns.regplot`, o exemplo de scatter com linha de tendência do Capítulo 5.3 (educação vs. PIB per capita, ajustado manualmente com `np.polyfit`). Compare as duas abordagens: quantas linhas cada uma exige, e o que se ganha ou perde ao trocar uma pela outra.

Solução comentada:

```
# Versão matplotlib puro (Capítulo 5.3) — 6 linhas relevantes
z = np.polyfit(df["educacao"], df["pib_per_capita"], 1)
p = np.poly1d(z)
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(df["educacao"], df["pib_per_capita"], color="steelblue", s=80, alpha=0.7,
edgecolor="black")
ax.plot(df["educacao"], p(df["educacao"]), "r--", alpha=0.7, label="Tendência linear")
ax.legend()

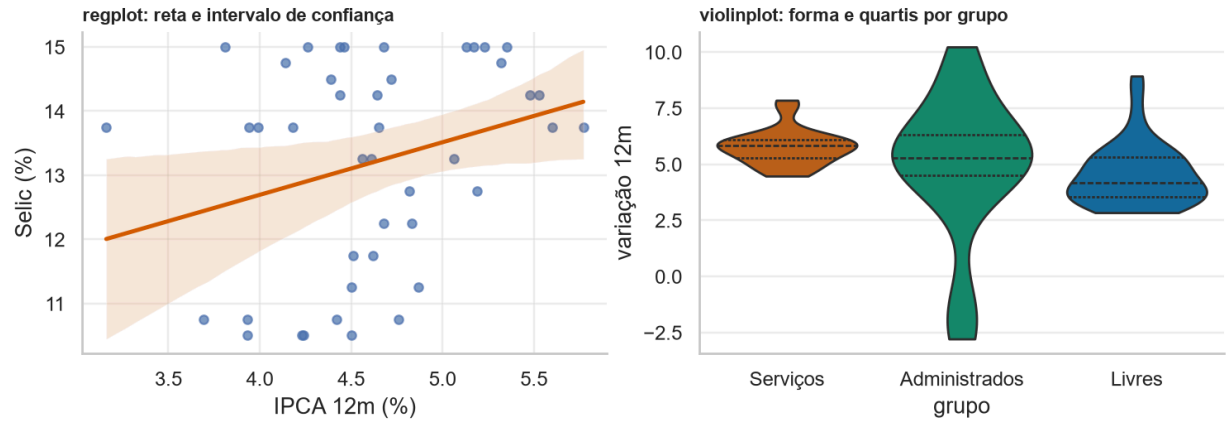
# Versão seaborn — 2 linhas relevantes, e já inclui intervalo de confiança
fig, ax = plt.subplots(figsize=(8, 6))
sns.regplot(data=df, x="educacao", y="pib_per_capita", ax=ax)
```

O ganho de seaborn aqui é duplo: menos código, e um recurso estatístico adicional (o intervalo de confiança sombreado) que a versão matplotlib manual não tinha, e que exigiria calcular manualmente o erro padrão da regressão para reproduzir. A perda é o controle fino: personalizar exatamente a cor de cada elemento, ou trocar o tipo de linha de tendência (por exemplo, uma regressão polinomial de grau 2 em vez de linear) exige consultar parâmetros específicos de `regplot` (`order=2`, por exemplo), enquanto na versão matplotlib manual qualquer mudança é direta, porque cada peça é escrita explicitamente. Regra prática: use seaborn para exploração rápida e para os tipos de gráfico estatístico que ele já cobre bem (regressão, distribuição, pares de variáveis); volte a matplotlib puro quando precisar de controle total sobre um gráfico final destinado a publicação.

Exemplo resolvido — seaborn: o mesmo gráfico com menos código
O `seaborn` constrói sobre o matplotlib gráficos estatísticos completos (regressão com intervalo de confiança, distribuições por grupo) em uma linha, com estilos agradáveis por padrão:

```
import seaborn as sns
longo = gr3[["Servicos", "Administrados", "Livres"]].rename(columns={"Servicos":
"Serviços"}).melt(var_name="grupo", value_name="variação 12m")
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9.5, 3.6))
sns.regplot(x=tay3["ipca12m"], y=tay3["selic"], ax=ax1, scatter_kws=dict(s=22, alpha=0.7),
line_kws=dict(color="#D55E00"))
ax1.set_title("regplot: reta e intervalo de confiança", loc="left", fontsize=10,
fontweight="bold")
ax1.set_xlabel("IPCA 12m (%)"); ax1.set_ylabel("Selic (%)")
sns.violinplot(data=longo, x="grupo", y="variação 12m", ax=ax2, inner="quartile", cut=0,
palette=["#D55E00", "#009E73", "#0072B2"], hue="grupo", legend=False)
ax2.set_title("violinplot: forma e quartis por grupo", loc="left", fontsize=10,
fontweight="bold")
for ax in (ax1, ax2):
    ax.spines[["top", "right"]].set_visible(False)
fig.text(0.01, 0.01, FONTE3, fontsize=7, color="gray")
fig.tight_layout(rect=(0, 0.04, 1, 1))
fig.savefig("figuras03/seaborn_exemplos.png", dpi=160)
plt.close(fig)
print("seaborn", sns.__version__, "| observações no formato longo:", len(longo))
```

```
seaborn 0.13.2 | observações no formato longo: 129
```



Fonte: Banco Central do Brasil (SGS); elaboração própria.

Figura 13: `regplot` e `violinplot` do seaborn sobre os mesmos dados reais.

Capítulo 14 — Salvando e exportando gráficos

14.1. Raster vs. vetorial: a distinção que mais importa

Formatos de imagem se dividem em duas famílias fundamentalmente diferentes, e escolher a errada é uma causa comum de gráficos borrados em impressões ou artigos:

Família	Formatos	Como funciona	Quando usar
Raster (matriz de pixels)	PNG, JPEG	Grade fixa de pixels coloridos — perde qualidade ao ampliar	Web, apresentações de slide, uso geral
Vetorial (fórmulas geométricas)	PDF, SVG, EPS	Descreve formas matematicamente — ampliação sem perda de qualidade	Artigos acadêmicos, impressão de alta qualidade, edição posterior

```
fig.savefig("grafico.png")                    # raster, padrão
fig.savefig("grafico.png", dpi=300)            # raster de alta resolução
fig.savefig("grafico.pdf")                     # vetorial — ideal para artigos
fig.savefig("grafico.svg")                     # vetorial — ideal para web/edição
```

Um artigo acadêmico submetido a uma revista quase sempre deveria usar PDF ou SVG para os gráficos, não PNG — porque o processo editorial frequentemente redimensiona figuras, e um PNG redimensionado para cima fica visivelmente pixelizado, enquanto um PDF permanece nítido em qualquer tamanho.

14.2. DPI: o que realmente significa

DPI (*dots per inch*, pontos por polegada) só é um conceito relevante para formatos **raster** — controla quantos pixels são gerados por polegada da figura final. Uma figura de `figsize=(10, 5)` (10 por 5 polegadas) salva com `dpi=100` gera uma imagem de 1000×500 pixels; a mesma figura salva com `dpi=300` gera 3000×1500 pixels — mais nítida, mas também um arquivo maior. Para formatos vetoriais (PDF, SVG), `dpi` é irrelevante para a nitidez das formas (que são sempre nítidas, por definição), afetando apenas elementos rasterizados embutidos, se houver (como uma imagem de fundo).

Uso pretendido	DPI recomendado
Visualização em tela, apresentação de slides	100-150
Impressão comum	200-300
Publicação acadêmica / revista	300+ (ou preferencialmente vetorial)

14.3. `bbox_inches="tight"` e recorte de bordas

Sem esse parâmetro, matplotlib às vezes corta rótulos de eixo ou legendas que se estendem além da área "padrão" da figura. `bbox_inches="tight"` recalcula automaticamente a área salva para incluir todos os elementos visíveis, com uma margem mínima:

```
fig.savefig("grafico.png", dpi=300, bbox_inches="tight")
```

14.4. Transparência de fundo

Para gráficos que serão inseridos sobre um fundo colorido (um slide de apresentação com tema escuro, por exemplo), `transparent=True` remove o fundo branco padrão, deixando apenas os elementos do gráfico visíveis:

```
fig.savefig("grafico_transparente.png", dpi=200, transparent=True)
```

14.5. Exemplo Resolvido — Escolhendo o formato certo para cada destino

Enunciado: você precisa entregar o mesmo gráfico de IPCA em três contextos: (1) um slide de apresentação em PowerPoint; (2) um artigo em LaTeX submetido a uma revista acadêmica; (3) uma postagem em uma página web institucional. Justifique o formato e os parâmetros de `savefig` para cada caso.

Solução comentada:

```
# (1) Slide de apresentação — raster é aceitável, resolução moderada, sem transparência
fig.savefig("ipca_slide.png", dpi=150, bbox_inches="tight")

# (2) Artigo acadêmico — vetorial, para permanecer nítido em qualquer tamanho de impressão
fig.savefig("ipca_artigo.pdf", bbox_inches="tight")

# (3) Página web — SVG mantém nitidez em qualquer zoom do navegador e tem arquivo leve
fig.savefig("ipca_web.svg", bbox_inches="tight")
```

O raciocínio: um slide é visualizado uma única vez, numa tela, geralmente sem zoom — um PNG de resolução moderada (150 DPI) é suficiente e mantém o arquivo leve. Um artigo acadêmico pode ser impresso ou ampliado por um revisor, e a maioria dos sistemas de submissão (incluindo LaTeX) trabalha nativamente com PDF vetorial, tornando-o a escolha natural. Uma página web se beneficia de SVG porque navegadores modernos renderizam SVG nativamente em qualquer resolução de tela (incluindo monitores de alta densidade de pixels) sem o peso de um PNG de altíssima resolução.

> **Aprofundando a teoria — o tamanho do arquivo em cada formato: a prova numérica**
> Formatos **raster** (PNG, JPG) guardam pixels: o tamanho cresce com a resolução (DPI) e perde nitidez ao ampliar. Formatos **vetoriais** (SVG, PDF) guardam instruções de desenho: escalam sem perda e costumam ser menores em gráficos simples. Para web e apresentações, PNG a 100–150 DPI; para impressão e artigos, PDF/SVG ou PNG a 300 DPI; fotos, JPG. O código mede:

```
import tempfile, os
fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(ip3.index, ip3.values); ax.set_title("IPCA 12m")
with tempfile.TemporaryDirectory() as pasta:
    for nome, kw in (("png_72dpi", dict(dpi=72, format="png")), ("png_150dpi",
dict(dpi=150, format="png")),
                     ("png_300dpi", dict(dpi=300, format="png")), ("svg",
dict(format="svg")), ("pdf", dict(format="pdf"))):
        caminho = os.path.join(pasta, "figura." + kw["format"])
        fig.savefig(caminho, **{k: v for k, v in kw.items() if k != "format"},
format=kw["format"])
        print(f"{nome:<11}: {os.path.getsize(caminho) / 1024:7.1f} KB")
plt.close(fig)
```

```
png_72dpi  :    15.9 KB
png_150dpi :    39.9 KB
png_300dpi :    86.0 KB
svg        :    20.2 KB
pdf        :     8.6 KB
```

Capítulo 15 — Boas práticas: checklist final de um gráfico econômico

Antes de considerar qualquer gráfico deste material pronto, uma revisão final combinando os princípios dos capítulos anteriores:

- **O tipo de gráfico corresponde à natureza dos dados?** Linha para série temporal contínua, barra para categorias, scatter para relação entre duas variáveis (Capítulos 2, 4, 5).
- **O eixo de barras começa em zero?** (Capítulo 4.3, 10.2)
- **Existe algum eixo duplo (`twinx`) cujos limites foram escolhidos de forma neutra, e não para "encaixar" visualmente duas séries?** (Capítulos 3.2, 10.5)
- **A paleta de cor é apropriada ao tipo de dado** (categórica, sequencial, divergente) **e segura para daltônicos?** (Capítulo 9)
- **Existe redundância de codificação** (cor + posição, cor + forma) **em pontos centrais do argumento?** (Capítulo 9.4)
- **O título comunica uma conclusão proporcional ao que os dados realmente mostram** — nem genérico demais, nem exagerado? (Capítulos 10.8, 11.1)
- **A fonte dos dados e a data de extração estão indicadas?** (Capítulo 11.3)
- **O formato de exportação é adequado ao destino final** (slide, artigo, web)? (Capítulo 14)

Pergunta do leitor	Gráfico adequado
Como o valor evoluiu no tempo?	Linha (poucas séries) ou linhas pequenas múltiplas
Qual categoria é maior?	Barras ordenadas, a partir de zero
Como se distribui?	Histograma (uma variável) ou boxplot/violino (vários grupos)
Duas variáveis se relacionam?	Scatter (com tendência, sem inferir causa)
Como se compõe o total?	Barras empilhadas 100% (poucas categorias); evite pizza
Quais pares de variáveis se correlacionam?	Heatmap divergente centrado em zero, ou `pairplot`
Qual a diferença entre dois momentos?	Barras lado a lado ou *slope chart*

Capítulo 16 — Exercícios para Executar (na mão)

Resolva no papel. **As soluções não estão neste documento** — quando terminar, peça para eu conferir suas respostas.

Exercício 1 — Identifique o erro de design
Um gráfico de barras compara o desemprego de 5 países. O eixo Y vai de 5% a 12% (não começa em zero), e as cores das barras são escolhidas aleatoriamente, uma por país, sem relação com o valor. Liste todos os problemas de design que você consegue identificar, usando os critérios do Capítulo 15.

Exercício 2 — Escolha o tipo de gráfico
Para cada situação abaixo, diga qual tipo de gráfico (linha, barra, scatter, histograma, boxplot, heatmap) é o mais adequado, e justifique com base na natureza do eixo X e na hierarquia de Cleveland-McGill:

a) Evolução do PIB trimestral do Brasil nos últimos 10 anos.
b) Comparação do PIB per capita entre 15 países em um único ano.
c) Relação entre gasto público em educação e taxa de alfabetização, para 40 países.
d) Distribuição de renda de uma amostra de 5000 domicílios.
e) Correlação entre 6 indicadores macroeconômicos diferentes.
f) Comparação da distribuição de salários entre 4 setores da economia.

Exercício 3 — Preveja antes de rodar
Sem rodar no computador, descreva como ficaria visualmente o gráfico gerado pelo código abaixo — em particular, o que acontece com a legibilidade das barras dado o número de categorias e a ausência de ordenação:

```
paises = ["Brasil", "Zimbábue", "Argentina", "Bolívia", "Chile", "Uruguai",
          "Colômbia", "Peru", "Equador", "Paraguai", "Venezuela", "Guiana"]
pib = [2.2, 4.1, -1.8, 1.5, 3.1, 3.4, 2.8, 2.5, 1.9, 3.9, -8.5, 5.2]

fig, ax = plt.subplots(figsize=(6, 3))
ax.bar(paises, pib)
```

Exercício 4 — Diagnóstico de gráfico enganoso
Um artigo apresenta dois gráficos de linha com eixos Y independentes (`twinx`) mostrando "Gastos com propaganda" e "Vendas da empresa", com uma escala escolhida de forma que as duas curvas parecem andar perfeitamente juntas ao longo de 5 anos. Explique por que essa aparência de correlação perfeita deveria ser questionada, mesmo sem acesso aos dados brutos.

Exercício 5 — Corrija a paleta
Um heatmap de correlação entre 8 variáveis macroeconômicas usa o colormap `jet`, sem `vmin`/`vmax` definidos. Explique os dois problemas técnicos disso (um de percepção, um de comparabilidade) e proponha a correção, sem escrever código — apenas descrevendo o raciocínio.

Exercício 6 — Redesenhando um título
Reescreva os três títulos abaixo para que sejam descritivos (contem a história dos dados) sem serem exagerados ou avaliativos além do que os números sustentam:

a) "IPCA 2015-2020" (dado: inflação subiu de 6% para 10% em dois anos, depois caiu para 3%)
b) "Gráfico de câmbio" (dado: câmbio ficou estável ±2% o período inteiro)
c) "Desemprego" (dado: desemprego caiu de 14% para 8% ao longo de 4 anos)

Capítulo 17 — Exercícios para Executar (em código)

Implemente e execute cada um no seu editor. **As soluções não estão neste documento** — o objetivo é você rodar de verdade e ver o resultado; quando terminar, peça para eu revisar seu código.

Exercício 1 — Primeira figura orientada a objetos
Baixe o IPCA (código SGS 433) de 2018 até hoje usando `bcb.sgs`, crie a figura com `plt.subplots()`, plote a série, adicione título e rótulo do eixo Y, e salve como PNG com 150 DPI.

Exercício 2 — Dois eixos Y
Baixe IPCA (433) e SELIC (11) de 2015 a 2026 e plote os dois com dois eixos Y (`twinx`), com cores e rótulos de eixo consistentes entre a linha e o eixo correspondente.

Exercício 3 — Subplots como alternativa honesta
Recrie o Exercício 2 usando `subplots(2, 1, sharex=True)` em vez de `twinx()`. Compare visualmente as duas versões.

Exercício 4 — Barras com regra do eixo zero
Crie um gráfico de barras comparando o PIB de 10 países em 2025. Destaque em uma cor os que tiveram PIB negativo e em outra os positivos, com o eixo Y começando em zero.

Exercício 5 — Barras horizontais ordenadas
Refaça o Exercício 4 como barras horizontais, ordenadas do menor para o maior PIB.

Exercício 6 — Curva de Phillips
Gere um scatter plot de inflação vs. desemprego (dados simulados ou reais) com linha de tendência ajustada por `np.polyfit`.

Exercício 7 — Mesmo scatter com seaborn
Recrie o Exercício 6 usando `sns.regplot`, e compare a quantidade de código necessária.

Exercício 8 — Histograma com bins alternativos
Baixe o IPCA de 1995 até hoje e gere três histogramas lado a lado (`subplots(1, 3)`) usando `bins=10`, `bins=30` e `bins="fd"`. Compare a forma da distribuição percebida em cada um.

Exercício 9 — Histograma com média e mediana
Para a mesma série do Exercício 8, adicione linhas verticais de média e mediana, e escreva (em comentário no código) se a distribuição parece simétrica ou assimétrica.

Exercício 10 — Histograma e boxplot lado a lado
Monte uma figura com `subplots(1, 2)`: histograma à esquerda, boxplot à direita, da mesma série do IPCA.

Exercício 11 — Boxplot comparando países
Simule (com `np.random.normal`) a inflação de 4 países com médias e desvios-padrão diferentes, e compare as distribuições com um boxplot único. Identifique, olhando o resultado, se algum país "achata" visualmente os outros por causa da escala.

Exercício 12 — Heatmap de correlação correto
Monte um `DataFrame` com pelo menos 5 variáveis macroeconômicas simuladas, calcule a matriz de correlação, e desenhe um heatmap com colormap divergente (`RdBu_r`), `vmin=-1`, `vmax=1`, e os valores numéricos escritos dentro de cada célula.

Exercício 13 — Heatmap com colormap errado (para comparar)
Refaça o Exercício 12 usando `cmap="jet"` sem `vmin`/`vmax`, e compare visualmente com a versão correta. Escreva, em comentário, qual das duas comunica melhor a diferença entre correlação positiva e negativa.

Exercício 14 — Redundância de codificação
Crie um scatter de PIB (eixo X) vs. inflação (eixo Y) para vários países, onde os países em recessão (PIB negativo) usem simultaneamente uma cor e um marcador diferentes dos países em expansão (não apenas cor).

Exercício 15 — Gráfico de barras enganoso vs. honesto
Gere dois gráficos de barra com os mesmos dados: um com o eixo Y cortado (não começando em zero) e outro com o eixo Y começando em zero. Coloque os dois lado a lado (`subplots(1, 2)`) para comparar a diferença de impressão visual.

Exercício 16 — Anotando um evento histórico
Baixe o câmbio USD/BRL (código SGS 1) desde 2010, plote a série, e adicione uma anotação (`axvline` + `annotate`) marcando um evento relevante (por exemplo, início da pandemia em março de 2020).

Exercício 17 — Gráfico publication-ready completo
Combine em um único gráfico: preenchimento sob a curva (`fill_between`), remoção de spines superior e direita, grade sutil atrás dos dados (`set_axisbelow`), formatação do eixo X por ano (`ticker.DateFormatter`), e nota de fonte no rodapé da figura.

Exercício 18 — Bubble chart com área corrigida
Crie um gráfico de bolhas (scatter com `s=` variável) representando PIB (eixo X), inflação (eixo Y) e população (tamanho da bolha) de 6 países. Certifique-se de escalar o tamanho da bolha pela **área**, não pelo raio (ou seja, `s` proporcional ao valor, não ao valor ao quadrado).

Exercício 19 — Exportando para três destinos
Gere um único gráfico e salve-o três vezes: como PNG a 150 DPI (para slide), como PDF vetorial (para um artigo), e como SVG (para uma página web). Compare o tamanho dos três arquivos gerados.

Exercício 20 — Pairplot exploratório
Monte um `DataFrame` com 4 variáveis macroeconômicas simuladas (PIB, inflação, desemprego, câmbio) para 100 observações e gere um `sns.pairplot()`. Identifique visualmente, olhando os scatters fora da diagonal, se alguma dupla de variáveis parece ter relação não linear.

Exercício 21 — Projeto integrador: painel de 4 gráficos
Usando `subplots(2, 2)`, monte um painel com: (1) linha do IPCA; (2) barras do PIB por país; (3) scatter de inflação vs. desemprego; (4) heatmap de correlação entre 4 variáveis. Aplique pelo menos três boas práticas do Capítulo 15 ao painel inteiro (título consistente, cores coerentes, fonte dos dados).

Capítulo Final — Resumo

Tipo de gráfico	Código	Uso principal	Cuidado central
Linha	`ax.plot(x, y)`	Séries temporais contínuas	Evitar "espaguete"; agregar/suavizar se necessário
Barra	`ax.bar(x, y)` / `ax.barh(x, y)`	Comparar categorias discretas	Eixo sempre começa em zero
Dispersão	`ax.scatter(x, y)`	Relação entre duas variáveis	Correlação não implica causalidade
Histograma	`ax.hist(x, bins=)`	Distribuição de uma variável	Escolha de `bins` altera a impressão
Boxplot	`ax.boxplot(x)`	Resumo estatístico / comparar grupos	IQR, outliers, cuidado com escalas muito diferentes
Heatmap	`ax.imshow(matriz, cmap=)`	Matriz de correlação	Paleta divergente + `vmin`/`vmax` fixos
Dois eixos Y	`ax.twinx()`	Séries com unidades diferentes	Limites arbitrários podem sugerir correlação falsa
Subplots	`plt.subplots(n, m)`	Múltiplos gráficos	Alternativa honesta a `twinx()`
seaborn (eixo)	`sns.lineplot(..., ax=ax)`	Combina com `Axes` existente	Aceita `ax=`
seaborn (figura)	`sns.pairplot()`, `sns.relplot()`	Cria sua própria `Figure`	Não aceita `ax=`
Salvar (raster)	`fig.savefig("f.png", dpi=300)`	Slides, web geral	Perde nitidez ao ampliar
Salvar (vetorial)	`fig.savefig("f.pdf")`	Artigos, impressão	Nítido em qualquer tamanho

Hierarquia de Cleveland-McGill (da mais para a menos precisa): posição > comprimento > inclinação > ângulo > área > volume > cor.

Checklist de honestidade visual (Capítulo 15): eixo zero em barras, limites neutros em `twinx`, paleta apropriada ao tipo de dado, redundância de codificação, título proporcional aos dados, fonte citada.

Capítulo 18 — Erros Comuns e Teste Rápido

18.1. Os erros de design que mais prejudicam um gráfico econômico

#	Erro	Como evitar
1	Barras com eixo Y cortado	Comece em zero; se for linha, deixe o corte evidente
2	Dois eixos Y com escalas arbitrárias	Mesma escala, índice base 100 ou subplots
3	Mais de 5 séries coloridas em um painel	Destaque uma e deixe as outras em cinza; ou small multiples
4	Pizza com muitas fatias	Barras ordenadas
5	Colormap arco-íris (`jet`)	Sequencial (magnitude) ou divergente simétrica (sinal)
6	Depender só da cor	Traço, marcador e rótulo direto
7	Título que descreve em vez de concluir	Título-conclusão
8	Esquecer unidades, fonte e período	Rótulo de eixo com unidade e nota de fonte
9	Barras em ordem alfabética	Ordene por valor
10	Salvar como JPG ou em baixa resolução	PNG ≥ 150 DPI ou PDF/SVG

18.2. Teste rápido: verdadeiro ou falso (com gabarito)

1. **V ou F:** Em um gráfico de barras, o eixo Y pode começar em qualquer valor. **Falso.** O comprimento codifica o valor; deve partir de zero.
2. **V ou F:** Um eixo Y duplo permite comparar duas séries de unidades diferentes sem distorcer. **Falso.** A escala de cada eixo é arbitrária.
3. **V ou F:** Para uma matriz de correlação, a escala de cor ideal é divergente e simétrica. **Verdadeiro.**
4. **V ou F:** Mais bins sempre dão um histograma mais fiel. **Falso.** Bins demais criam ruído.
5. **V ou F:** O boxplot mostra se a distribuição é bimodal. **Falso.** Comprime a forma; use histograma ou violino.
6. **V ou F:** Uma linha de tendência no scatter prova causalidade. **Falso.** Descreve associação.
7. **V ou F:** Cor, sozinha, deve ser a única forma de distinguir as séries. **Falso.** Acrescente traço, marcador e rótulo.
8. **V ou F:** Um PDF vetorial perde nitidez ao ampliar. **Falso.** Escala sem perda.
9. **V ou F:** Ordenar barras por valor facilita a comparação. **Verdadeiro.**
10. **V ou F:** `jet` é uma boa escolha para dados que devem ser lidos por daltônicos. **Falso.**

Próxima apostila

Quando terminar os exercícios acima, siga para a **Apostila 4 — Econometria para Economia**. Lá, os gráficos de dispersão com linha de tendência do Capítulo 5 e os intervalos de confiança do `sns.regplot` (Capítulo 13) deixam de ser apenas elementos visuais e passam a ser formalizados estatisticamente — regressão linear, testes de hipótese, séries temporais e os pressupostos que sustentam (ou não) a interpretação causal que este material, no Capítulo 5.3, alertou para não presumir apressadamente.